Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:
«Разработка базы данных медицинской клиники»

Студент ИУ7-62Б _____ К. А. Атаев
(Группа) (Подпись, дата) (И. О. Фамилия)

Руководитель курсовой работы _____ В. И. Солодовников
(Подпись, дата) (И. О. Фамилия)

2026 г.

РЕФЕРАТ

Расчетно-пояснительная записка содержит 65 страниц, 22 рисунка, 16 таблиц, 1 приложение, 10 источников.

Ключевые слова: медицинская клиника, база данных, PostgreSQL, Redis, Go, Gin, React, TypeScript, UUID, триггеры, хранимые процедуры, индексы, конкурентный доступ.

Объектом разработки является информационная система медицинской клиники, обеспечивающая регистрацию пациентов, ведение расписания врачей, запись на прием, проведение приема, формирование счетов и учет оплаты услуг.

Цель работы — спроектировать и реализовать базу данных и приложение доступа к ней для автоматизации деятельности медицинской клиники, а также исследовать вопросы согласованности данных и производительности ключевых операций.

В ходе выполнения работы проведены анализ предметной области медицинской клиники, формализация задачи, проектирование структуры базы данных, ролевой модели и прикладного интерфейса доступа. Реализованы схема базы данных PostgreSQL, хранимые процедуры, триггеры, сценарии тестирования и кэширование расписания врачей и прайс-листа услуг с использованием Redis. Разработан backend на Go и Gin с архитектурой Repository $\rightarrow$ Service $\rightarrow$ Handler и подготовлен frontend на React и TypeScript. Выполнены проверки валидации, HTTP-уровня, кэширования, конкурентного доступа и производительности ключевых операций.

СОДЕРЖАНИЕ

РЕФЕРАТ 4

ВВЕДЕНИЕ 8

1 Аналитический раздел 10

1.1 Анализ предметной области и существующих решений 10

1.2 Формализация задачи 11

1.3 Формализация данных и описание структуры базы данных 12

1.4 Формализация ролей 14

1.5 Модели данных 15

1.5.1 Иерархическая модель 15

1.5.2 Сетевая модель 16

1.5.3 Реляционная модель 16

1.5.4 Многомерная модель 16

1.5.5 Объектно-ориентированная модель 17

1.5.6 Выбор модели данных 17

1.6 Вывод 18

2 Конструкторский раздел 19

2.1 Схема проектируемой базы данных 19

2.2 Описание сущностей 21

2.2.1 Подсистема безопасности 21

2.2.2 Справочные сущности 21

2.2.3 Медицинские сущности 22

2.2.4 Сущности расписания и записи 22

2.2.5 Финансовые сущности 23

2.3 Ограничения целостности 23

2.3.1 Домены и перечислимые типы 23

2.3.2 Ключи и связи многие-ко-многим 24
2.3.3 Ограничения времени и запрет конфликтов 24
2.3.4 Доказательство 3НФ 25
2.4 Процедуры, функции и триггеры 25
2.4.1 Вспомогательные функции 25
2.4.2 Процедуры делового регламента 27
2.4.3 Функции пересчета счета и триггеры 32
2.5 Ролевая модель базы данных 34
2.6 Транзакционная стратегия и требования к кэшированию 35
2.7 Вывод 36
3 Технологический раздел 37
3.1 Средства реализации 37
3.2 Состав реализованной базы данных 38
3.3 Реализованные ограничения целостности 39
3.4 Реализованные функции и триггеры 40
3.5 Ролевая модель базы данных 41
3.6 Разработка интерфейса доступа к базе данных 42
3.7 Порядок тестирования и классы эквивалентности 46
3.8 Вывод 48
4 Исследовательский раздел 49
4.1 Описание исследования 49
4.2 Результаты исследования 50
4.3 Анализ результатов 54
4.4 Вывод 55
ЗАКЛЮЧЕНИЕ 57
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 58

ПРИЛОЖЕНИЕ А 59

ВВЕДЕНИЕ

Актуальность темы курсовой работы обусловлена необходимостью автоматизации деятельности медицинской клиники, в которой ежедневно выполняются регистрация пациентов, формирование расписания врачей, запись на прием, проведение приема, расчет стоимости услуг и учет оплат. Для такой предметной области особенно важны непротиворечивость данных, защита от конфликтов по времени, сохранение истории изменений и разграничение прав доступа.

Объектом разработки является информационная система медицинской клиники. Предметом разработки являются структура базы данных, механизмы обеспечения целостности, прикладной интерфейс доступа к данным и кэширование часто используемой информации.

Целью курсовой работы является проектирование и реализация базы данных медицинской клиники и приложения доступа к ней, обеспечивающих корректную запись на прием, проведение приема, формирование счетов, учет оплат и кэширование прайс-листа и расписания врачей.

Для достижения поставленной цели необходимо решить следующие задачи:

1) провести анализ предметной области медицинской клиники и существующих подходов к ее автоматизации;
2) формализовать задачу разработки базы данных и приложения;
3) определить сущности предметной области, связи между ними и ограничения целостности;
4) спроектировать ролевую модель базы данных, процедуры, триггеры и транзакционные сценарии;
5) выбрать средства реализации базы данных, backend- и frontend-частей приложения;
6) реализовать структуру базы данных, серверную часть, кэширование в Redis и демонстрационный пользовательский интерфейс;
7) разработать тесты для проверки записи, отмены, проведения приема, формирования счетов и регистрации оплаты;
8) исследовать конкурентный доступ и влияние индексов на производи-

тельность ключевых операций.

1 Аналитический раздел

В этом разделе проводится анализ предметной области медицинской клиники, формализуется задача разработки базы данных и приложения, выделяются основные сущности и роли пользователей, а также рассматриваются модели данных, пригодные для хранения и обработки медицинской информации.

1.1 Анализ предметной области и существующих решений

Предметная область охватывает деятельность медицинской клиники, в которой требуется хранить сведения о пациентах и врачах, управлять расписанием приема, оформлять запись, проводить прием, рассчитывать стоимость услуг и регистрировать оплаты. Особенностью данной области является необходимость одновременного соблюдения медицинских, организационных и финансовых требований: система должна быть удобной для повседневной работы персонала, но при этом не должна допускать конфликтов по времени, произвольного изменения стоимости услуг и нарушения согласованности между приемом, счетом и оплатой.

Основные проблемы предметной области можно сформулировать следующим образом:

— расписание врача задается интервалами доступности и должно исключать пересечение записей;
— запись на прием выполняется несколькими пользователями одновременно, поэтому система должна корректно работать при конкурентном доступе;
— стоимость медицинских услуг может изменяться во времени, следовательно, требуется хранение истории цен;
— завершение приема должно приводить к формированию счета на основе реально оказанных услуг, а не только предварительной записи;
— доступ к операциям должен различаться в зависимости от роли пользователя.

Среди существующих решений можно выделить три класса систем: специализированные медицинские информационные системы, универсальные

CRM-платформы и ручной учет в электронных таблицах. Медицинские информационные системы обычно уже содержат расписание врачей, карточки пациентов, приемы и оплату услуг, но их внутренняя структура часто закрыта, а настройка под конкретные правила клиники требует внедрения готового продукта. Универсальные CRM удобны для учета клиентов и задач, однако медицинские сущности в них приходится выражать через сделки, статусы и пользовательские поля. Электронные таблицы подходят для простого учета, но не защищают данные от ошибок при одновременной работе нескольких пользователей.

Сравнение существующих решений приведено в таблице 1.1.

Таблица 1.1 – Сравнение существующих решений для автоматизации клиники

Критерий	Медицинские ИС	Универсальные CRM	Электронные таблицы
Хранение расписания врачей	Штатный календарь врача и кабинета.	Календарь и задачи без медицинских правил.	Таблица расписания, заполняемая вручную.
Запрет пересечения записей	Обычно реализован на уровне системы.	Требует настройки правил или доработки.	Контролируется пользователем вручную.
История цен услуг	Поддерживается через прайс-листы и периоды действия.	Может храниться в пользовательских полях.	Есть риск перезаписи старых значений.
Проведение приема	Есть карточка приема, диагноз и рекомендации.	Не является базовой сущностью CRM.	Обычно ведется на отдельных листах.
Поддержка счетов и оплат	Счета, строки счетов и статусы оплаты.	Частично через сделки, счета или интеграции.	Ручной расчет и ручное изменение статусов.
Ролевая модель	Пациент, врач, администратор и другие роли.	Роли интерфейса без явной модели прав БД.	Ограничение доступа только к файлу или листу.
Аудит и проверка данных	Журнал действий и отчеты.	Частично зависит от настроек CRM.	Проверка затруднена, ошибки легко внести вручную.

В рамках курсовой работы необходимо разработать собственную базу данных, которая хранит данные медицинской клиники и поддерживает основные операции: запись на прием, проведение приема, формирование счетов и регистрацию оплат. При этом часть правил должна контролироваться самой базой данных: запрет пересечения записей врача, хранение истории цен, пересчет состояния счета и разделение прав доступа.

1.2 Формализация задачи

В ходе выполнения курсовой работы необходимо разработать базу данных для хранения информации о пациентах, врачах, специальностях, медицинских услугах, расписании приема, записях, приемах, счетах и оплатах медицинской клиники. Также необходимо спроектировать и разработать приложение, которое будет предоставлять интерфейс для взаимодействия с базой данных.

Необходимо предусмотреть возможность регистрации пациентов, просмотра расписания врачей и прайс-листа, записи пациента на прием, отмены

записи, проведения приема врачом, внесения результатов приема, формирования счета по оказанным услугам и регистрации оплаты. Для разных категорий пользователей требуется реализовать собственный набор прав.

Пациент должен иметь возможность работать со своими записями и просматривать доступную информацию. Врач должен иметь возможность просматривать свое расписание, проводить прием и вносить его результаты. Администратор должен иметь возможность управлять справочниками, расписанием, записями пациентов и оплатами.

При разработке необходимо обеспечить корректность хранения данных. Запись на прием не должна пересекаться с уже занятыми интервалами врача, стоимость услуги должна сохраняться с учетом истории изменения цен, а состояние счета должно обновляться после изменения оказанных услуг или оплат. В качестве основной СУБД используется PostgreSQL, для кэширования часто запрашиваемых данных используется Redis.

1.3 Формализация данных и описание структуры базы данных

На основании формализации задачи выделены следующие основные сущности предметной области: пользователь, роль, специальность, врач, пациент, услуга, цена услуги, интервал доступности врача, отсутствие врача, запись на прием, прием, оказанная услуга, счет и оплата. Для описания связи многие-ко-многим между врачами и услугами используется отдельная сущность связи.

На основании выделенных сущностей построена ER-диаграмма, представленная на рисунке 1.1. Диаграмма отражает основные сущности предметной области и связи между ними: пользователи связаны с ролями через промежуточную таблицу, врачи привязаны к специальностям и услугам, записи на прием связывают пациента с врачом и услугой, а завершение приема порождает счет и оплаты.

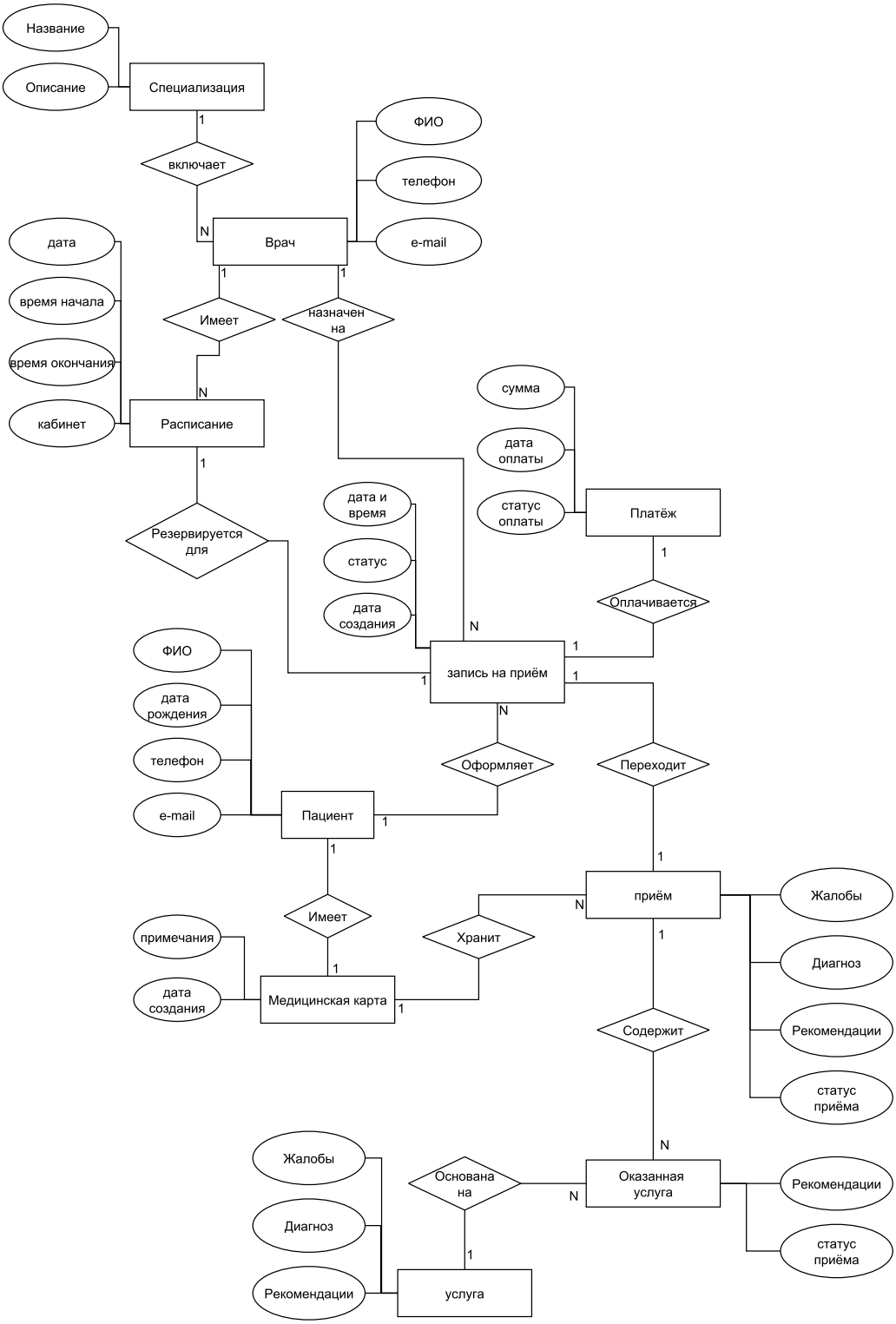


Рисунок 1.1 – ER-диаграмма сущностей базы данных медицинской клиники

Таким образом, база данных должна обслуживать не только хранение справочной информации, но и сквозной процесс оказания медицинской услуги: от записи пациента на прием до формирования счета и регистрации оплаты.

1.4 Формализация ролей

В системе выделяются три прикладные роли:

1) пациент;
2) администратор;
3) врач.

Для каждой роли определен собственный набор полномочий. Пациент регистрирует собственную карточку, выбирает врача и услугу, оформляет запись и при необходимости отменяет ее с учетом установленных ограничений. Администратор управляет пользователями, ролями, справочниками и финансовыми операциями, а также сопровождает запись пациента. Врач работает со своим расписанием, проводит прием и фиксирует его результаты.

Полномочия ролей приведены в таблице 1.2.

Таблица 1.2 – Роли пользователей и их основные полномочия

Роль	Основные действия	Ограничения
Пациент	регистрация собственной карточки, запись и отмена приема, просмотр доступного расписания и прайс-листа	не проводит прием, не регистрирует оплату и не управляет ролями
Администратор	управление пользователями, ролями, справочниками, просмотр счетов, регистрация оплат и сопровождение записи пациента	не проводит прием
Врач	просмотр своего расписания, завершение приема, фиксация диагноза и оказанных услуг	не регистрирует оплаты и не управляет пользователями

На рисунке 1.2 приведена диаграмма вариантов использования, отражающая основные действия ролей в системе.



Рисунок 1.2 – Диаграмма вариантов использования системы медицинской клиники

Ролевая структура напрямую влияет на проектирование прикладного API и прав доступа в базе данных, поэтому в конструкторском разделе она будет детализирована уже в терминах ролей PostgreSQL и доступных операций.

1.5 Модели данных

1.5.1 Иерархическая модель

Иерархическая модель данных представляет базу как дерево, в котором каждая запись, кроме корневой, имеет единственного родителя [1]. Классическим примером является СУБД IBM IMS, разработанная в 1960-х годах для хранения спецификаций космической программы Apollo. Запросы в иерархической модели выполняются навигационным способом — путем обхода дерева от корня к потомкам.

Для предметной области медицинской клиники такой подход неудобен: пациент, врач, услуга, прием и счет образуют сеть взаимосвязанных сущностей,

а не простую древовидную иерархию. Например, один пациент может иметь несколько записей к разным врачам, а один врач — несколько услуг, что невозможно выразить единственным деревом без дублирования данных.

1.5.2 Сетевая модель

Сетевая модель, формализованная в стандарте CODASYL [2], позволяет записи иметь несколько родительских и дочерних записей, что снимает ограничение иерархической модели. Примером является СУБД IDMS. Навигация по данным осуществляется через явные указатели между записями.

Однако проектирование и сопровождение сетевой модели значительно сложнее, а ее практическое применение в современных прикладных системах ограничено. Изменение структуры связей требует модификации схемы на физическом уровне, что делает модель негибкой для развивающейся системы клиники.

1.5.3 Реляционная модель

Реляционная модель, предложенная Э. Ф. Коддом в 1970 году [1], хранит данные в виде отношений (таблиц), связанных между собой ключами. Она естественно поддерживает нормализацию, ограничения целостности, индексы, транзакции, представления, процедуры и триггеры [2]. Примеры современных реляционных СУБД: PostgreSQL, MySQL, Oracle Database, Microsoft SQL Server.

Для медицинской клиники это особенно важно, так как требуется одновременно обеспечить целостность расписания, истории цен, приемов, счетов и оплат. Декларативный язык SQL позволяет формулировать сложные запросы без навигационного обхода данных.

1.5.4 Многомерная модель

Многомерная модель ориентирована на аналитические и OLAP-задачи, когда данные рассматриваются как факты и измерения [2]. Типичная реализация — куб данных с осями «время», «услуга», «врач». Примеры СУБД: Microsoft Analysis Services, ClickHouse (колоночное хранение).

Для основной транзакционной системы клиники такая модель непригод-

на, поскольку операции записи, отмены, проведения приема и оплаты требуют именно оперативной обработки (OLTP) и строгих ограничений целостности, которые многомерные СУБД не обеспечивают.

1.5.5 Объектно-ориентированная модель

Объектно-ориентированная модель хранит данные в виде объектов с инкапсулированными методами и поддерживает наследование [1]. Примерами являются СУБД ObjectDB и db4o. Такая модель удобна при тесной интеграции с объектными языками программирования.

Однако в рамках данной задачи она не дает достаточных преимуществ перед реляционной моделью. Контроль целостности временных интервалов (EXCLUDE-ограничения), кэширование и конкурентный доступ в курсовом проекте должны опираться на развитые возможности реляционной СУБД, которых объектно-ориентированные СУБД в полной мере не предоставляют.

1.5.6 Выбор модели данных

Сравнение рассмотренных моделей данных по критериям, существенным для разрабатываемой системы, приведено в таблице 1.3.

Таблица 1.3 – Сравнение моделей данных

Критерий	Иерарх.	Сетевая	Реляц.	Многомер.	ОО
Связи M:N	Нет	Да	Да	Нет	Да
Декларативные запросы	Нет	Нет	Да	Частично	Нет
Ограничения целостности	Частично	Частично	Да	Нет	Частично
Транзакционная обработка	Частично	Частично	Да	Нет	Да
Распространенность СУБД	Низкая	Низкая	Высокая	Средняя	Низкая

Наиболее подходящей для разрабатываемой системы является реляционная модель данных. Она обеспечивает естественное выражение связей между сущностями, поддержку нормализации и надежную транзакционную обработку. Выбор PostgreSQL как реляционной СУБД [3] дополнительно позволяет использовать перечислимые типы, домены, диапазоны времени, ограничения `EXCLUDE` [5], процедуры и триггеры, что полностью соответствует требованиям задания.

1.6 Вывод

В аналитическом разделе исследована предметная область медицинской клиники, формализована задача разработки базы данных и приложения, выделены сущности и роли пользователей, а также рассмотрены основные модели данных. По результатам анализа обоснован выбор реляционной модели и сформирована основа для конструкторского проектирования базы данных, процедур, ролевой модели и механизмов кэширования.

2 Конструкторский раздел

В данном разделе разработана структура базы данных медицинской клиники, определены ограничения целостности, процедуры, триггеры, роли и правила кэширования.

2.1 Схема проектируемой базы данных

База данных разделена на логические схемы безопасности, справочников, расписания, медицинских и финансовых данных. Такое разбиение позволяет отделить подсистему безопасности от справочных, операционных и финансовых данных. ER-диаграмма сущностей приведена в аналитическом разделе (рисунок 1.1).

Схема проектируемой базы данных приведена на рисунке 2.1.

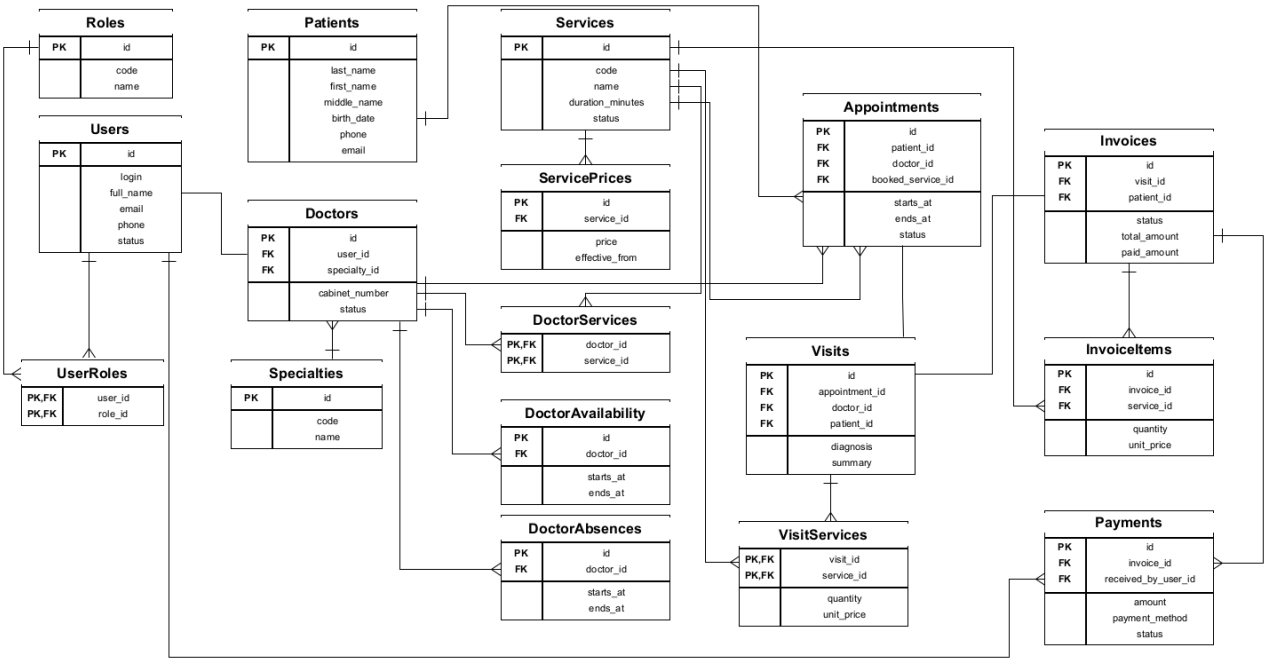


Рисунок 2.1 – Схема проектируемой базы данных медицинской клиники

2.2 Описание сущностей

В разработанной базе данных сущности сгруппированы по функциональным схемам PostgreSQL. Такое разбиение соответствует фактической реализации проекта и позволяет разделить подсистемы безопасности, справочников, медицинских данных, расписания и расчетов.

2.2.1 Подсистема безопасности

Таблица `security.roles` хранит перечень ролей базы данных прикладного уровня. Для каждой роли задаются уникальные атрибуты `code` и `name`, позволяющие однозначно идентифицировать назначение роли.

Таблица `security.users` описывает учетные записи пользователей системы. В ней хранятся логин, хэш пароля, полное имя, контактные данные и статус учетной записи. Данная таблица используется для учетных записей прикладных пользователей системы: пациента, администратора и врача, а также для фиксации автора операций записи, отмены и приема оплаты.

Таблица `user_roles` задает связь «пользователь – роль». Через нее одному пользователю могут быть назначены несколько ролей.

2.2.2 Справочные сущности

Таблица `catalog.specialties` содержит справочник врачебных специальностей. Записи этой таблицы используются при описании врачей и при группировке расписания и отчетов по профилю врача.

Таблица `catalog.services` описывает медицинские услуги, доступные в клинике. Для услуги хранятся код, наименование, описание, нормативная длительность приема и статус доступности. Именно эта сущность используется как при планировании записи, так и при формировании фактически оказанных услуг.

Таблица `catalog.service_prices` хранит историю стоимости услуг. Для каждой цены задаются дата начала действия, дата окончания действия и стоимость в валюте `RUB`. Таким образом обеспечивается возможность расчета стоимости услуг с учетом периода действия прайс-листа.

2.2.3 Медицинские сущности

Таблица `medical.patients` хранит сведения о пациентах: фамилию, имя, отчество, дату рождения, контактные данные, номер страхового полиса и статус записи о пациенте. Эта таблица используется при записи на прием, ведении истории обращений и формировании счетов.

Таблица `medical.doctors` описывает врачей клиники. Каждая запись связана с учетной записью пользователя и со специальностью, а также содержит номер кабинета и статус врача. Связь с таблицей `security.users` имеет характер один-к-одному, что позволяет разделить сведения о сотруднике и сведения о доступе в систему.

Таблица `doctor_services` реализует связь многие-ко-многим между врачами и услугами. Она определяет, какие услуги конкретный врач имеет право выполнять.

Таблица `medical.visits` фиксирует факт проведенного приема. Она связана с конкретной записью на прием, а также содержит ссылки на врача и пациента, диагноз и краткое описание результата приема.

Таблица `medical.visit_services` содержит перечень услуг, фактически оказанных в рамках приема. Для каждой позиции сохраняются количество, цена за единицу и итоговая стоимость строки, что позволяет далее автоматически формировать счет.

2.2.4 Сущности расписания и записи

Таблица `scheduling.doctor_availability` описывает интервалы доступности врача для записи. Для каждого интервала задаются начало и окончание периода, длительность слота и признак активности расписания.

Таблица `scheduling.doctor_absences` хранит интервалы отсутствия врача с указанием причины. Эти данные используются при контроле доступности времени записи и при проверке возможности проведения приема.

Таблица `scheduling.appointments` описывает записи пациентов на прием. Она связывает пациента, врача и планируемую услугу, хранит время начала и окончания приема, статус записи, служебное примечание, причину отмены, а также ссылки на пользователей, создавших или отменивших запись. Данная таблица является центральной сущностью операционного контура базы

данных.

2.2.5 Финансовые сущности

Таблица `billing.invoices` хранит счета, сформированные по результатам проведенного приема. Каждый счет связан с одним приемом и пациентом, а также содержит итоговую сумму, сумму оплаты и текущий статус расчета.

Таблица `billing.invoice_items` содержит позиции счета. Каждая позиция связывает счет с конкретной услугой и хранит количество, цену и стоимость строки. Содержимое данной таблицы формируется на основе таблицы `medical.visit_services`.

Таблица `billing.payments` хранит сведения об оплатах по счетам. Для каждой оплаты фиксируются сумма, способ оплаты, статус, внешний идентификатор операции и пользователь, зарегистрировавший оплату.

Таким образом, основная цепочка предметной области в реализованной базе данных выглядит следующим образом: пациент или администратор формирует запись пациента к врачу на услугу, проведенный прием порождает запись о визите и перечень оказанных услуг, после чего на их основе формируется счет и регистрируются оплаты.

2.3 Ограничения целостности

2.3.1 Домены и перечислимые типы

В схеме используются домены PostgreSQL `security.email_address` и `security.phone_number`. Они задают единые правила проверки e-mail и телефона и позволяют централизованно использовать один формат в таблицах пользователей и пациентов.

Для описания состояний сущностей применяются перечислимые типы PostgreSQL. Это исключает появление произвольных строковых значений и фиксирует допустимые переходы между состояниями. Модель статусов приведена в таблице 2.1.

Таблица 2.1 – Модель статусов и перечислимых типов PostgreSQL

Сущность	Тип PostgreSQL	Допустимые значения
Пользователь	security. user_status	active, inactive, archived
Пациент	medical. patient_status	active, inactive, archived
Врач	medical. doctor_status	active, inactive, archived
Услуга	catalog. service_status	active, inactive, archived
Запись на прием	scheduling. appointment_status	booked, cancelled, completed
Счет	billing. invoice_status	issued, partially_paid, paid, cancelled
Оплата	billing. payment_status	completed, cancelled

2.3.2 Ключи и связи многие-ко-многим

Во всех основных таблицах в качестве первичных ключей используются значения типа UUID. Такой подход позволяет формировать идентификаторы независимо от конкретного клиента. Он также не раскрывает информацию о количестве записей.

Ссылочная целостность обеспечивается внешними ключами. Связи многие-ко-многим реализованы тремя промежуточными таблицами:

— doctor_services — связь «врач – услуга»;
— visit_services — связь «прием – оказанная услуга»;
— user_roles — связь «пользователь – роль».

2.3.3 Ограничения времени и запрет конфликтов

Для таблиц истории цен, доступности врача, отсутствий врача и записей на прием используются диапазоны типа TSTZRANGE. Это позволяет единообразно хранить временные интервалы и применять к ним ограничения пересечения.

Для предотвращения двойной записи у одного врача используется ограничение EXCLUDE. Оно запрещает пересечение интервалов времени у активных записей и тем самым обеспечивает бесконфликтную работу в многопользова-

тельском режиме.

2.3.4 Доказательство 3НФ

Спроектированная база данных находится в третьей нормальной форме по следующим причинам:

1) все таблицы имеют первичный ключ, а их атрибуты являются атомарными;
2) в таблицах со составным ключом, таких как `doctor_services`, `visit_services` и `user_roles`, неключевые атрибуты не зависят от части ключа;
3) транзитивные зависимости устранены выделением самостоятельных сущностей для ролей, специальностей, истории цен, оказанных услуг, счетов и оплат.

Следовательно, каждый неключевой атрибут зависит только от ключа, от полного ключа и ни от чего, кроме ключа, что и соответствует требованиям 3НФ.

2.4 Процедуры, функции и триггеры

На уровне базы данных реализованы процедуры делового регламента и вспомогательные функции. Они обеспечивают контроль прав, выбор актуальной цены, пересчет счета и автоматическое обновление служебных полей.

2.4.1 Вспомогательные функции

Функция `set_updated_at` используется триггерами для автоматического обновления поля `updated_at` перед изменением записи. Схема функции приведена на рисунке 2.2.



Рисунок 2.2 – Схема функции `set_updated_at`

Функция `has_role` проверяет наличие прикладной роли у пользователя. Схема функции приведена на рисунке 2.3.

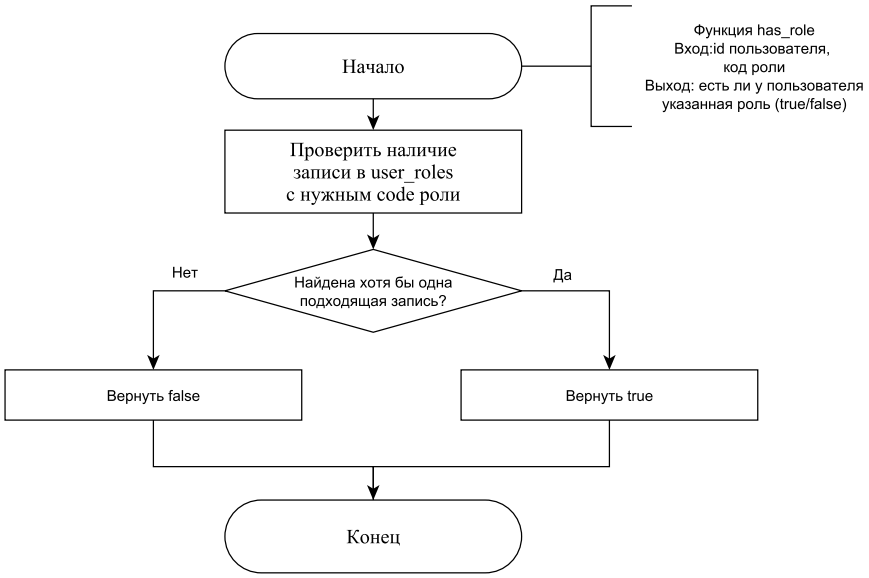


Рисунок 2.3 – Схема функции `has_role`

Функция `get_service_price` выбирает актуальную цену услуги на заданный момент времени. Схема функции приведена на рисунке 2.4.



Рисунок 2.4 – Схема функции `get_service_price`

2.4.2 Процедуры делового регламента

Схема процедуры записи на прием приведена на рисунке 2.5.

Функция `book_appointment` проверяет время начала приема, активность пациента и врача, закрепление услуги за врачом и активность самой услуги. Затем вычисляется время окончания приема, проверяются интервал доступности врача и отсутствие записи о его отсутствии. Конфликт по времени дополнительно предотвращается ограничением `EXCLUDE` при вставке записи.

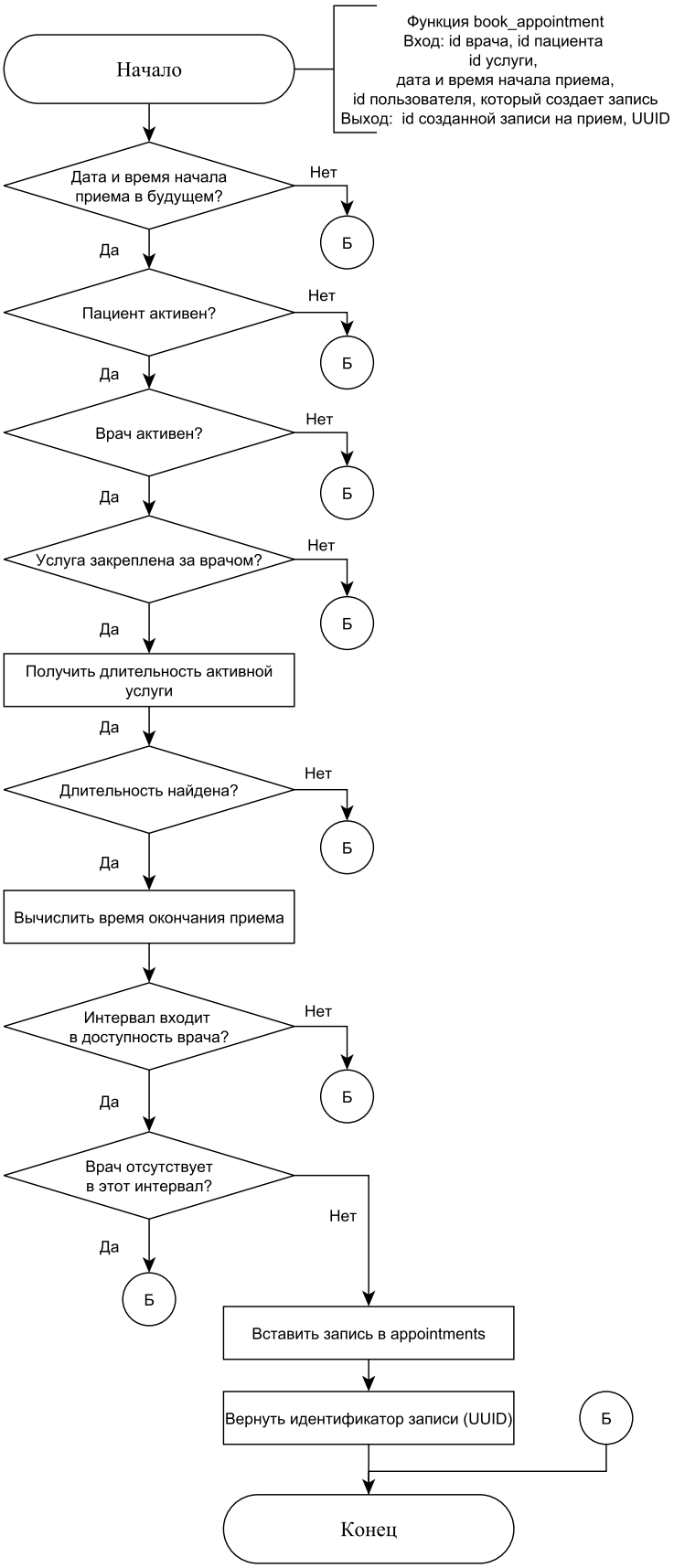


Рисунок 2.5 – Схема функции `book_appointment`

Схема отмены записи приведена на рисунке 2.6.

Функция `cancel_appointment` блокирует строку записи и проверяет ее состояние. Для пациента дополнительно действует правило отмены не позднее чем за 24 часа до приема, тогда как администратор может выполнить отмену без этого ограничения.

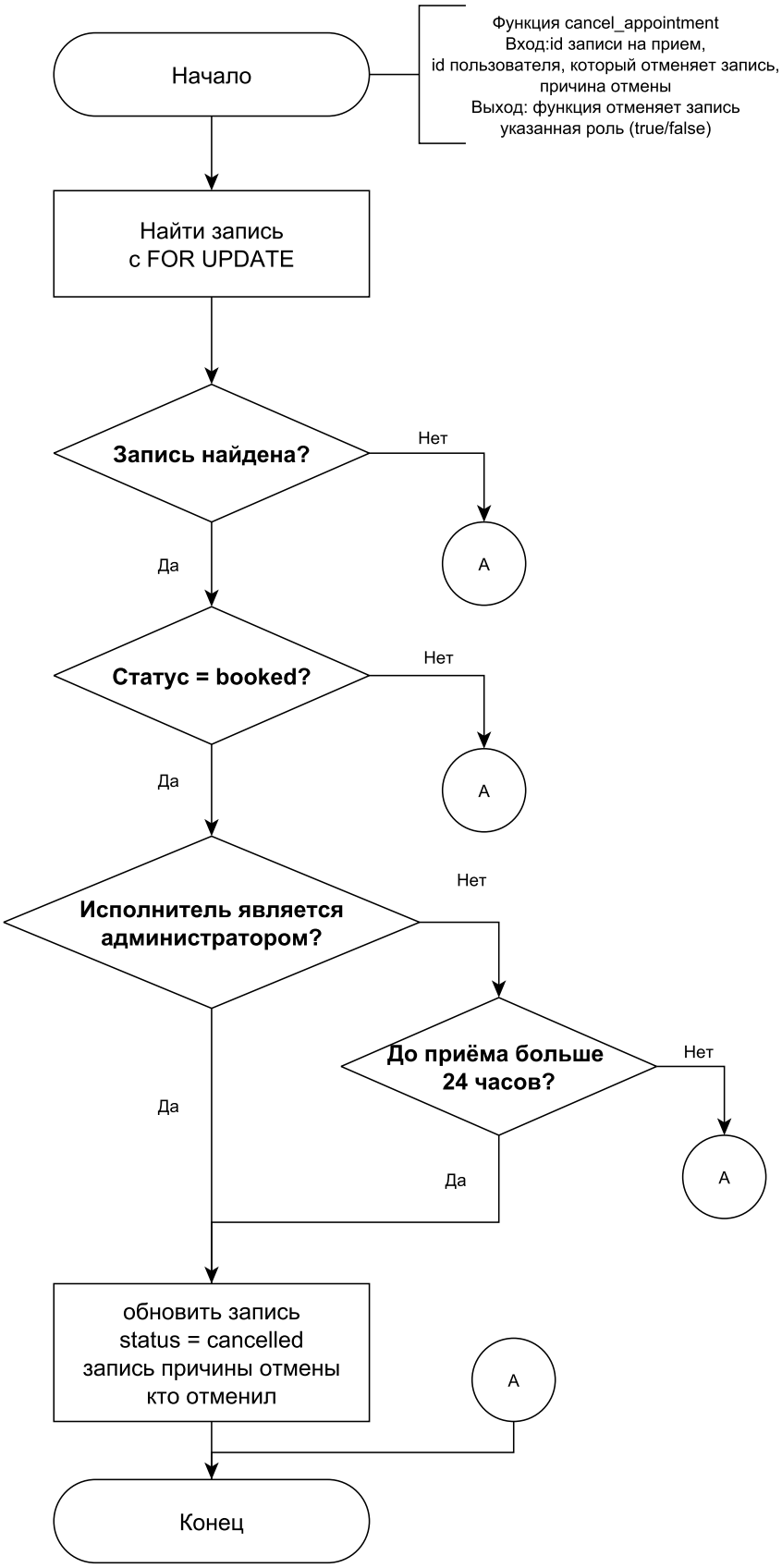


Рисунок 2.6 – Схема функции `cancel_appointment`

Схема функции проведения приема приведена на рисунке 2.7.

Функция `complete_visit` создает запись о визите и фиксирует оказанные услуги. Если список оказанных услуг пуст, в `visit_services` заносится забронированная услуга; иначе обрабатывается переданный список услуг. Затем формируются счет и строки счета, запись на прием переводится в состояние `completed`, после чего вызывается `refresh_invoice_state`.

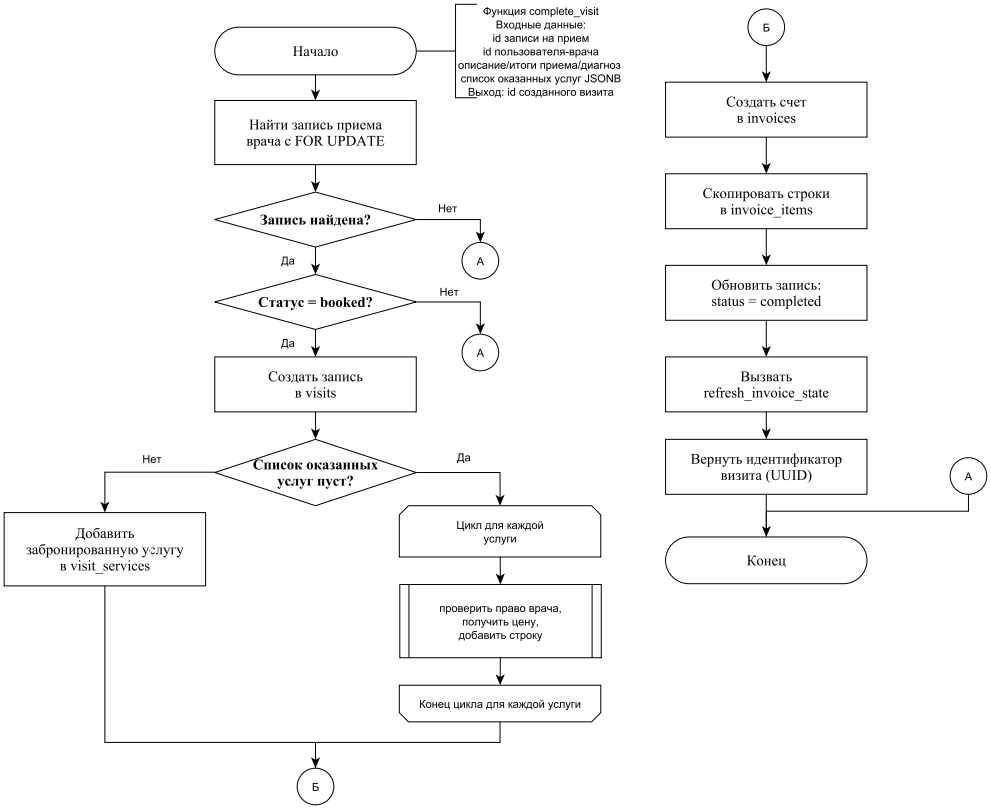


Рисунок 2.7 – Схема функции `complete_visit`

Схема регистрации оплаты приведена на рисунке 2.8.

Функция `register_payment` создает запись об оплате. Затем инициируется пересчет состояния счета.

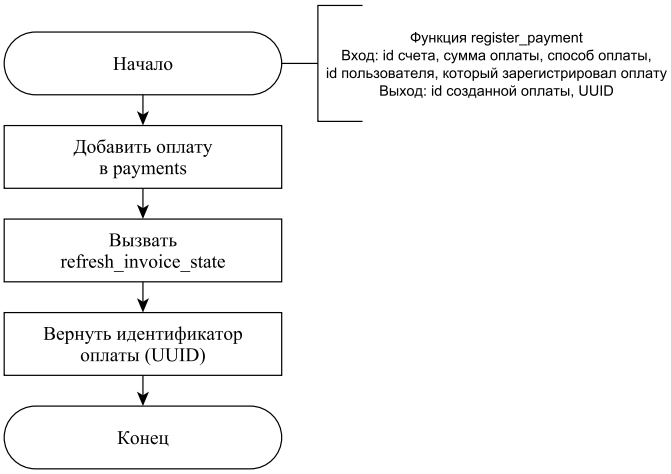


Рисунок 2.8 – Схема функции `register_payment`

2.4.3 Функции пересчета счета и триггеры

Схема функции `refresh_invoice_state` приведена на рисунке 2.9.

Функция `refresh_invoice_state` вычисляет итоговую сумму счета, сумму проведенных оплат и итоговый статус расчета.

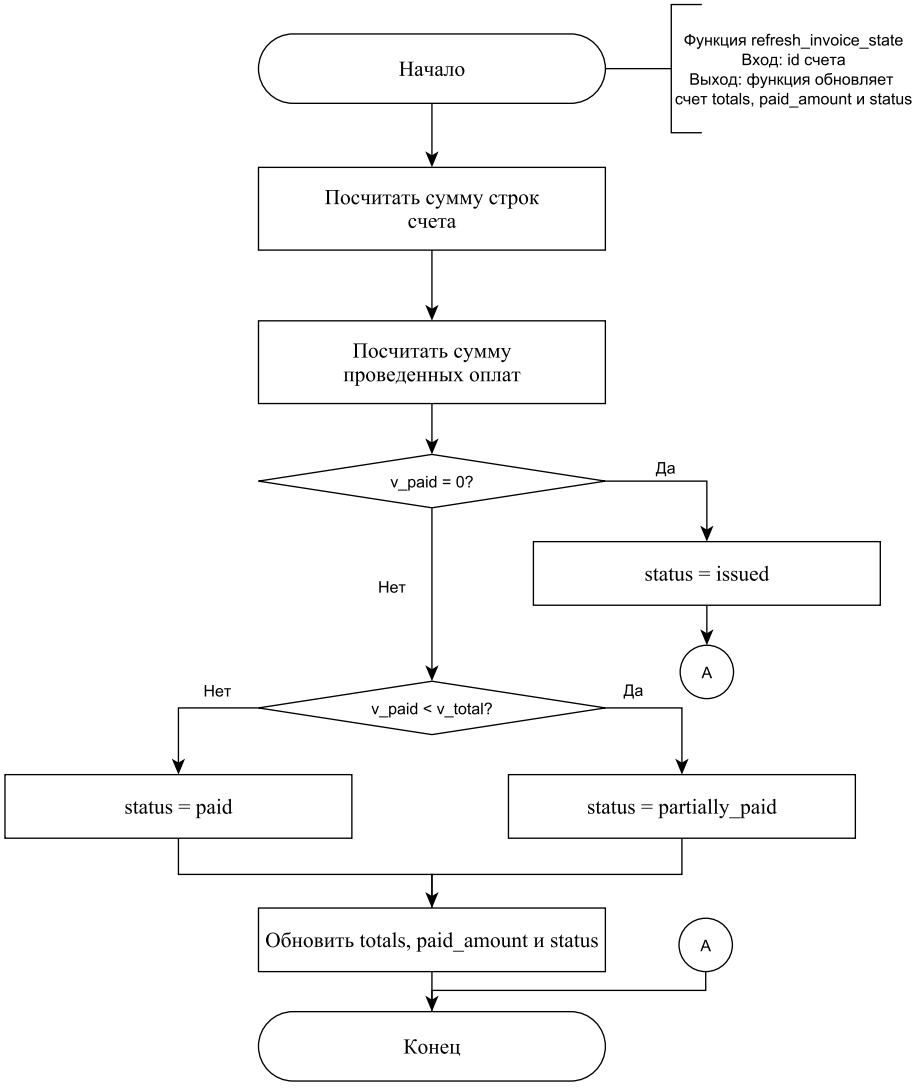


Рисунок 2.9 – Схема функции `refresh_invoice_state`

Схема триггеров пересчета счета приведена на рисунке 2.10.

Триггеры `trg_invoice_items_refresh_invoice` и `trg_payments_refresh_invoice` срабатывают после изменения строк счета или оплат, определяют идентификатор счета и вызывают функцию `refresh_invoice_state`.

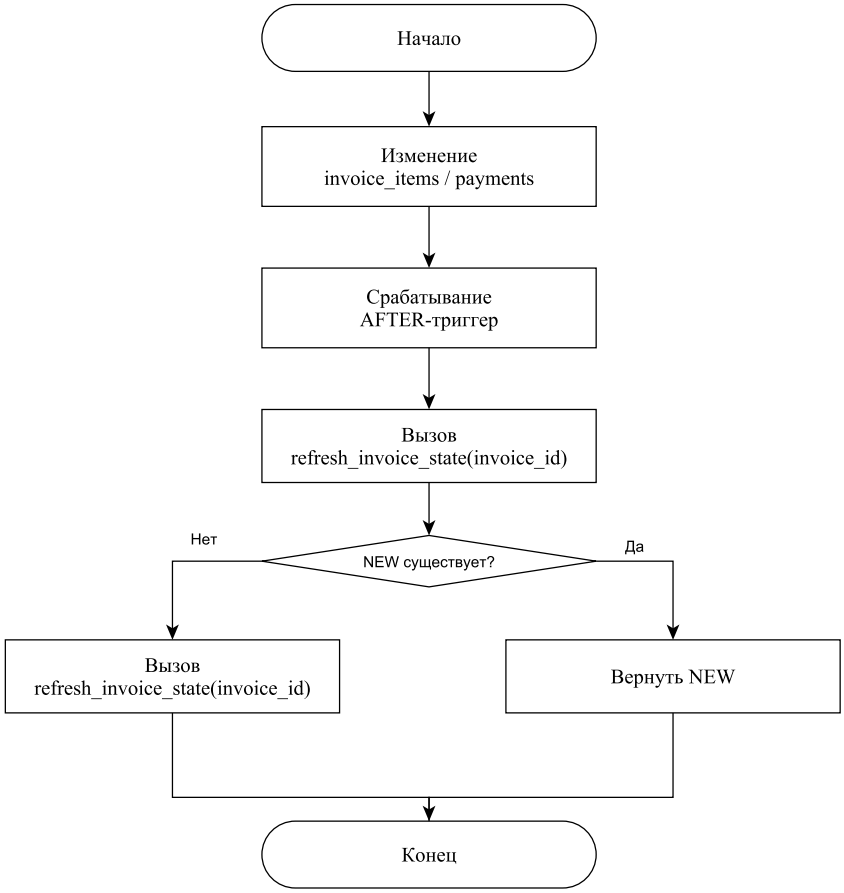


Рисунок 2.10 – Схема триггеров пересчета счета

Перечень триггеров базы данных приведен в таблице 2.2.

Таблица 2.2 – Триггеры базы данных

Имя триггера	Таблица и событие	Назначение
`trg__ updated_at`	`users, patients, doctors, services, appointments, visits, invoices, doctor_availability, doctor_absences;` событие `BEFORE UPDATE`	Автоматическое обновление поля `updated_at`
`trg_invoice_items_ refresh_invoice`	`billing.invoice_items;` событие `AFTER INSERT, UPDATE, DELETE`	Пересчет суммы и статуса счета
`trg_payments_ refresh_invoice`	`billing.payments;` событие `AFTER INSERT, UPDATE, DELETE`	Пересчет суммы оплат и статуса счета

2.5 Ролевая модель базы данных

Следует различать прикладные роли системы и роли базы данных.

Прикладные роли описывают, кто работает в интерфейсе системы, например пациент, администратор или врач. Ролевая модель базы данных описывает роли PostgreSQL и выданные им привилегии на объекты схемы.

В конструкторском разделе требуется именно ролевая модель на уровне базы данных.

В реализованной схеме используются три роли PostgreSQL:

— `clinic_owner`;

— `clinic_api`;

— `clinic_reporting`.

Их назначение приведено в таблице 2.3.

Таблица 2.3 – Ролевая модель базы данных

Роль PostgreSQL	Назначение	Предоставляемые права
`clinic_owner`	Владелец схемы базы данных	Создание и изменение схем, таблиц, индексов, процедур, триггеров и ролей
`clinic_api`	Роль backend-приложения	USAGE на схемы; SELECT, INSERT, UPDATE, DELETE для рабочих таблиц; EXECUTE для процедур и функций
`clinic_reporting`	Роль отчетов и аналитики	Доступ только на чтение к данным, необходимым для формирования отчетов

Прикладные роли пользователей не дублируются отдельными SQL-ролями. Они контролируются на уровне приложения и работают через backend, использующий роль `clinic_api`.

Таким образом, разграничение доступа на уровне интерфейса и на уровне СУБД остаются разделенными и не смешиваются в одной модели.

2.6 Транзакционная стратегия и требования к кэшированию

Критические операции — запись на прием, отмена записи, проведение приема и регистрация оплаты — выполняются в транзакционном контексте.

Контроль согласованности сосредоточен в PostgreSQL. Он обеспечивается сочетанием транзакций, ограничений целостности, диапазонных типов и исключающих ограничений.

Для ускорения чтения в Redis кэшируются два объекта:

— текущий прайс-лист услуг по ключу `clinic:catalog:pricelist` с временем жизни 600 секунд;
— расписание врача по ключу вида `clinic:schedule:doctor:...` с временем жизни 600 секунд.

Кэш расписания инвалидируется после записи на прием, отмены записи и проведения приема. Кэш прайс-листа обновляется при повторном чтении после истечения времени жизни записи.

2.7 Вывод

В конструкторском разделе спроектирована структура базы данных медицинской клиники, доказано соответствие 3НФ, описаны ограничения целостности, процедуры, функции, триггеры, ролевая модель базы данных и правила кэширования. Полученная схема соответствует требованиям проекта и обеспечивает основу для дальнейшей реализации приложения.

3 Технологический раздел

В данном разделе описаны выбранные средства реализации базы данных и приложения, состав реализованной базы данных, ограничения целостности, роли базы данных, интерфейс доступа к данным и порядок тестирования функций базы данных.

3.1 Средства реализации

Для реализации базы данных выбрана СУБД PostgreSQL. Данная СУБД поддерживает ACID-транзакции, домены, перечислимые типы, индексы, триггеры, хранимые функции и процедуры, диапазонные типы данных, а также ограничение EXCLUDE. Эти возможности позволяют перенести критические правила предметной области на уровень базы данных: запрет пересечения записей по времени, проверку актуальности цены, пересчет состояния счета и контроль допустимых статусов.

PostgreSQL также подходит для выбранной предметной области за счет развитой поддержки внешних ключей, транзакционной согласованности и диапазонных типов данных [4]. Расширение btree_gist используется для ограничения пересечения временных интервалов. Для генерации UUID применяется расширение pgcrypto.

Для серверной части выбраны язык Go [7], пакет database/sql, драйвер pgx и фреймворк Gin [8]. Такое сочетание позволяет реализовать REST API и разделить код backend-приложения на слои Handler, Service и Repository. При этом критическая бизнес-логика не дублируется в приложении, а вызывается через функции PostgreSQL.

В качестве программного интерфейса доступа к данным выбран стандартный интерфейс Go database/sql с драйвером pgx. Выбор обусловлен тем, что данный интерфейс соответствует стеку backend-приложения, поддерживает параметризованные SQL-запросы, транзакции, работу через пул соединений и прямой вызов функций PostgreSQL. Универсальные интерфейсы ODBC, ADO.NET и JDBC в данном проекте не выбраны, так как они ориентированы на другие технологические платформы или требуют лишнего промежуточного слоя. Пол-

ноценная ORM также не использована, поскольку в курсовой работе требуется явно показать работу с реализованными функциями, триггерами и ограничениями базы данных.

Для клиентской части выбраны React [9] и TypeScript [10]. Клиентское приложение используется как демонстрационный интерфейс для регистрации пациента, просмотра прайс-листа, просмотра расписания, записи на прием, отмены записи, проведения приема и регистрации оплаты.

Для кэширования часто запрашиваемых данных используется Redis [6]. В кэш помещаются актуальный прайс-лист услуг и представление расписания врача за выбранный интервал. Сводная информация о выбранных средствах приведена в таблице 3.1.

Таблица 3.1 – Выбранные средства реализации

Слой	Технология	Назначение
Frontend	React + TypeScript	Реализация пользовательского интерфейса
Backend	Go + Gin	Реализация REST API и серверной логики
Доступ к БД	database/sql + pgx	Выполнение SQL-запросов и вызов функций PostgreSQL
СУБД	PostgreSQL	Хранение данных, контроль целостности, индексы, функции и триггеры
InMemory СУБД	Redis	Кэширование прайс-листа и расписания врачей
API	REST	Обмен данными между frontend и backend

3.2 Состав реализованной базы данных

База данных реализована в сценарии db/schema.sql. В ней созданы логические схемы security, catalog, medical, scheduling и billing. Такое разбиение отделяет учетные записи и роли от справочников, медицинских данных, расписания и финансового контура.

Состав реализованных таблиц приведен в таблице 3.2.

Таблица 3.2 – Сущности реализованной базы данных

Схема	Реализованные сущности
security	roles, users, user_roles. Сущности отвечают за учетные записи, прикладные роли и связь многие-ко-многим между пользователями и ролями.
catalog	specialties, services, service_prices, представление current_price_list. Сущности описывают специальности, услуги и историю цен.
medical	patients, doctors, doctor_services, visits, visit_services. Сущности хранят данные пациентов, врачей, оказанных услуг и результатов приема.
scheduling	doctor_availability, doctor_absences, appointments. Сущности реализуют расписание, периоды отсутствия врача и записи пациентов на прием.
billing	invoices, invoice_items, payments. Сущности отвечают за формирование счетов, строки счетов и регистрацию оплат.

Связи многие-ко-многим реализованы через отдельные промежуточные таблицы. В качестве первичных ключей основных сущностей используются UUID. Фрагмент реализации сущности записи на прием приведен в листинге А.1 приложения А.

3.3 Реализованные ограничения целостности

В базе данных реализованы домены, перечислимые типы, первичные и внешние ключи, ограничения уникальности, проверки значений, диапазонные ограничения и индексы. Основные группы ограничений приведены в таблице 3.3.

Таблица 3.3 – Реализованные ограничения целостности

Группа ограничений	Реализация
Форматы контактных данных	Домены security.email_address и security.phone_number проверяют формат e-mail и телефона.
Состояния сущностей	Перечислимые типы user_status, patient_status, doctor_status, service_status, appointment_status, invoice_status, payment_status.
Ссылочная целостность	Внешние ключи связывают пользователей с ролями, врачей со специальностями, записи с пациентами и врачами, визиты с записями, счета с визитами, оплаты со счетами.
Уникальность	Уникальны логины пользователей, коды ролей, коды услуг, названия специальностей, связь врача с учетной записью, связь визита с записью и связь счета с визитом.
Числовые проверки	Ограничения CHECK запрещают отрицательные суммы, нулевое количество услуг, некорректную длительность приема и дату рождения позже текущей даты.
Временные интервалы	Для цен, доступности, отсутствий и записей используются диапазоны TSTZRANGE; дополнительно проверяется, что окончание интервала позже начала.
Запрет пересечений	Ограничения EXCLUDE запрещают пересечение периодов цен одной услуги, активных интервалов доступности врача и активных записей одного врача.

Для ускорения ключевых операций созданы индексы по статусам пользователей, пациентов, врачей и услуг, по телефону пациента, по специальности врача, по истории цен, по расписанию врача, по истории пациента, по счетам пациента и по оплатам счета. Фрагменты реализации ограничений целостности приведены в листинге А.2 приложения А.

3.4 Реализованные функции и триггеры

На стороне базы данных реализованы функции, инкапсулирующие основные правила предметной области. Перечень функций и триггеров приведен в таблице 3.4. Схемы алгоритмов данных функций и триггеров приведены в конструкторском разделе на рисунках 2.2–2.10.

Таблица 3.4 – Реализованные функции и триггеры базы данных

Объект	Назначение
security. set_updated_at	Обновляет поле updated_at перед изменением записи.
security. has_role	Проверяет наличие прикладной роли у пользователя.
catalog. get_service_price	Возвращает актуальную цену услуги на заданный момент времени.
scheduling. book_appointment	Создает запись на прием после проверки пациента, врача, услуги, доступности врача и отсутствия врача.
scheduling. cancel_appointment	Отменяет запись с учетом статуса записи и роли пользователя, инициировавшего отмену.
medical. complete_visit	Создает визит, фиксирует оказанные услуги, формирует счет и переводит запись в состояние completed.
billing. register_payment	Регистрирует оплату и инициирует пересчет состояния счета.
billing. refresh_invoice_state	Пересчитывает итоговую сумму счета, сумму оплат и статус счета.
trg__updated_at	Триггеры автоматического обновления поля updated_at.
trg_invoice_items_ refresh_invoice, trg_payments_ refresh_invoice	Триггеры пересчета счета после изменения строк счета или оплат.

Реализация перечисленных объектов находится в сценарии db/schema.sql. Функции созданы операторами CREATE OR REPLACE FUNCTION, а автоматический вызов служебной логики обеспечивается триггерами CREATE TRIGGER. Функция получения актуальной цены услуги приведена в листинге А.3 приложения А, фрагмент функции записи на прием – в листинге А.4, а триггеры пересчета состояния счета – в листинге А.5.

3.5 Ролевая модель базы данных

На уровне PostgreSQL реализованы три роли базы данных. Они отделяют владение схемой, работу backend-приложения и доступ к отчетным данным.

Таблица 3.5 – Ролевая модель на уровне базы данных

Роль	Назначение	Права
clinic_owner	Владелец структуры базы данных	Создание и изменение схем, таблиц, индексов, функций, триггеров и ролей.
clinic_api	Роль backend-приложения	USAGE на схемы, SELECT, INSERT, UPDATE, DELETE на таблицы, EXECUTE на функции.
clinic_ reporting	Роль отчетов и аналитики	SELECT на таблицы и USAGE на схемы без прав изменения данных.

Прикладные роли пациента, администратора и врача хранятся в таблицах security.roles и security.user_roles. Они используются backend-приложением и функцией has_role для проверки бизнес-правил, например права администратора отменять запись без ограничения в 24 часа. Фрагмент SQL-сценария создания ролей и выдачи прав приведен в листинге А.6 приложения А.

3.6 Разработка интерфейса доступа к базе данных

Frontend не обращается к базе данных напрямую. Все операции выполняются через REST API backend-приложения. Backend-слой валидирует входные данные, вызывает репозитории и функции PostgreSQL, а затем возвращает результат клиенту в формате JSON.

Реализация доступа к данным построена по трехслойной схеме: Handler, Service и Repository. Обработчики HTTP-запросов отвечают за прием и валидацию DTO, сервисный слой выполняет прикладную координацию и работу с Redis-кэшем, а репозитории являются единственным слоем, напрямую выполняющим SQL-запросы через database/sql. Такой интерфейс отделяет клиентское приложение от конкретной СУБД и одновременно сохраняет явный доступ к функциям PostgreSQL, разработанным в рамках курсовой работы.

Компонентная структура приложения показана на рисунке 3.1. Пользователь работает с React-интерфейсом, frontend отправляет REST-запросы в backend, а backend последовательно передает обработку на уровни Handler, Service и Repository. Репозиторий обращается к PostgreSQL как к основной СУБД и к Redis как к InMemory-хранилищу кэшируемых представлений.

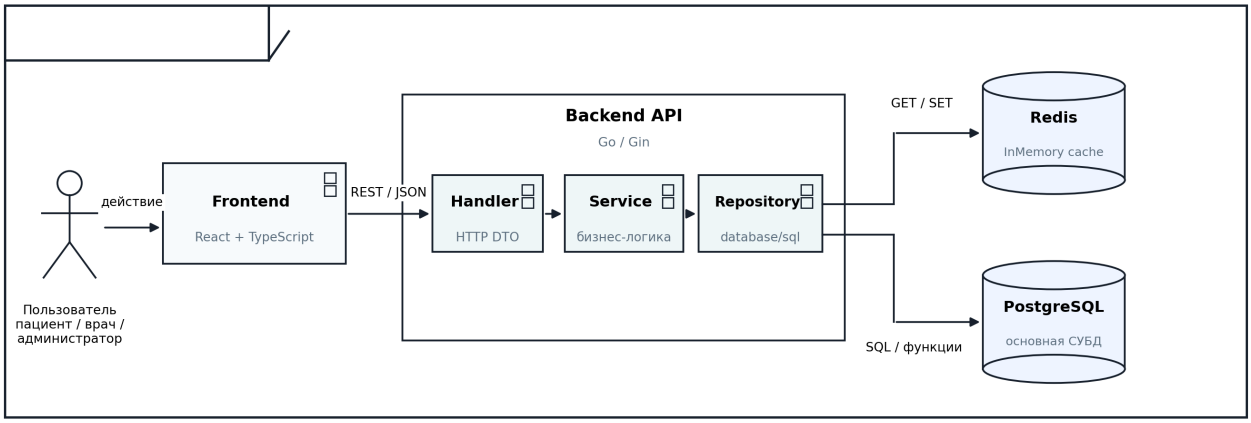


Рисунок 3.1 – UML-диаграмма компонентов приложения

Последовательность взаимодействия приложения с Redis и PostgreSQL при работе с кэшем показана на рисунке 3.2. На схеме отдельно приведены чтение данных через Redis-кэш и изменение данных с последующей инвалидацией кэша расписания. При отсутствии значения в Redis backend читает данные из PostgreSQL и сохраняет JSON-представление в кэш; при наличии значения результат возвращается без обращения к основной СУБД.

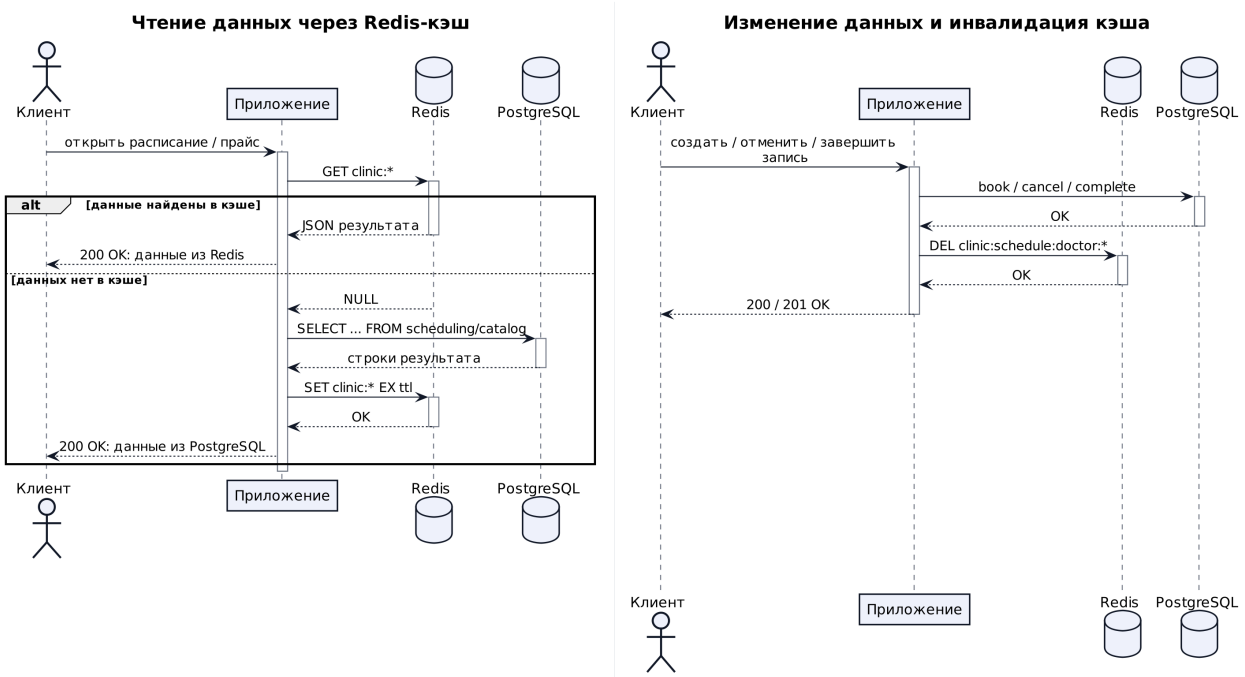


Рисунок 3.2 – UML-диаграмма последовательности доступа к PostgreSQL и Redis

Таблица 3.6 – Интерфейс доступа к базе данных

Маршрут REST API	Обращение к базе данных	Назначение
POST /api/v1/patients	medical.patients	Регистрация пациента.
GET /api/v1/services/ pricelist	catalog. current_price_list	Получение актуального прайс-листа с использованием Redis-кэша.
GET /api/v1/doctors/ :doctorId/schedule	doctor_availability, appointments	Получение расписания врача с использованием Redis-кэша.
POST /api/v1/appointments	scheduling. book_appointment	Запись пациента на прием.
POST /api/v1/appointments/ :id/cancel	scheduling. cancel_appointment	Отмена записи.
POST /api/v1/appointments/ :id/complete	medical. complete_visit	Проведение приема и формирование счета.
POST /api/v1/invoices/ :id/payments	billing. register_payment	Регистрация оплаты по счету.

В рамках курсовой работы подготовлен frontend-интерфейс для выполнения основных операций медицинской клиники. Авторизация выполняется по учетной записи пользователя: после входа backend возвращает роль и связанные идентификаторы пациента или врача. Экран входа показан на рисунке 3.3. Рабочее место после входа адаптируется под роль пользователя и показано на рисунке 3.4. Оформление записи на прием приведено на рисунке 3.5.

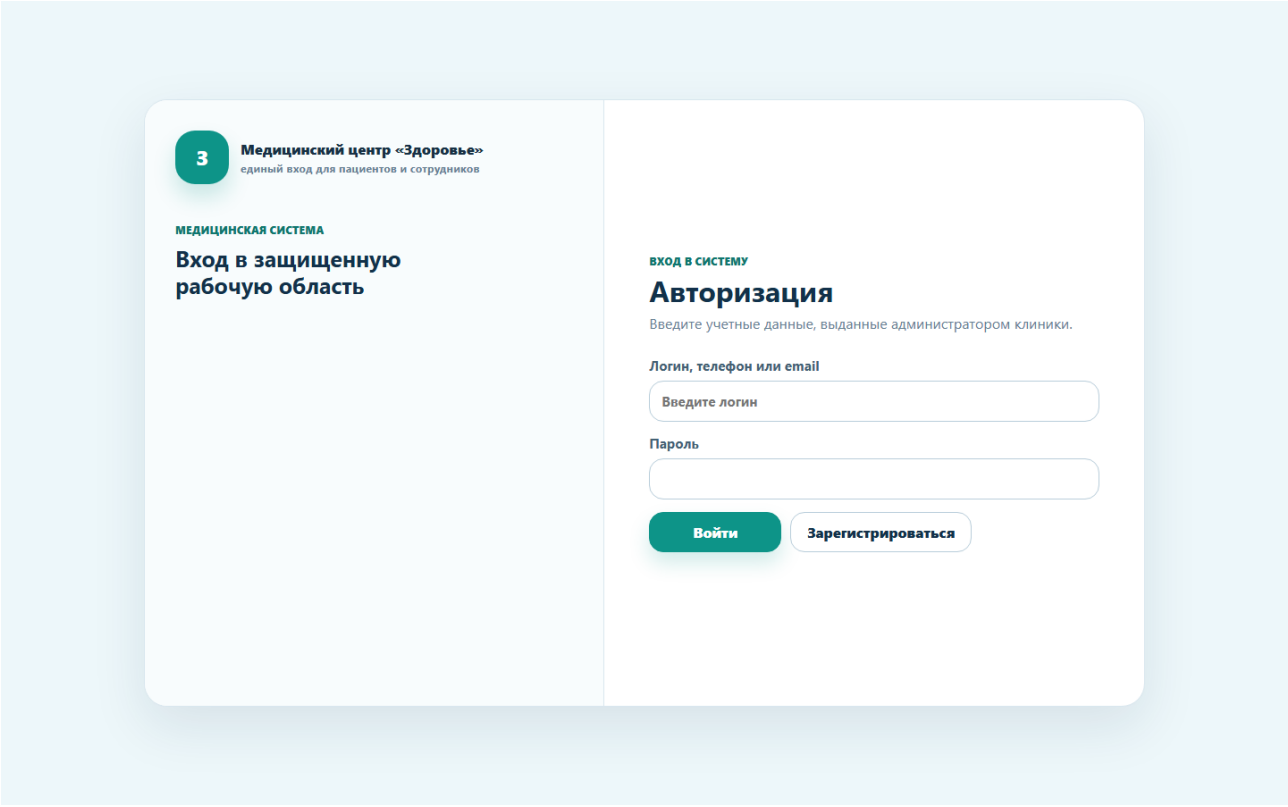


Рисунок 3.3 – Экран входа по пользователям

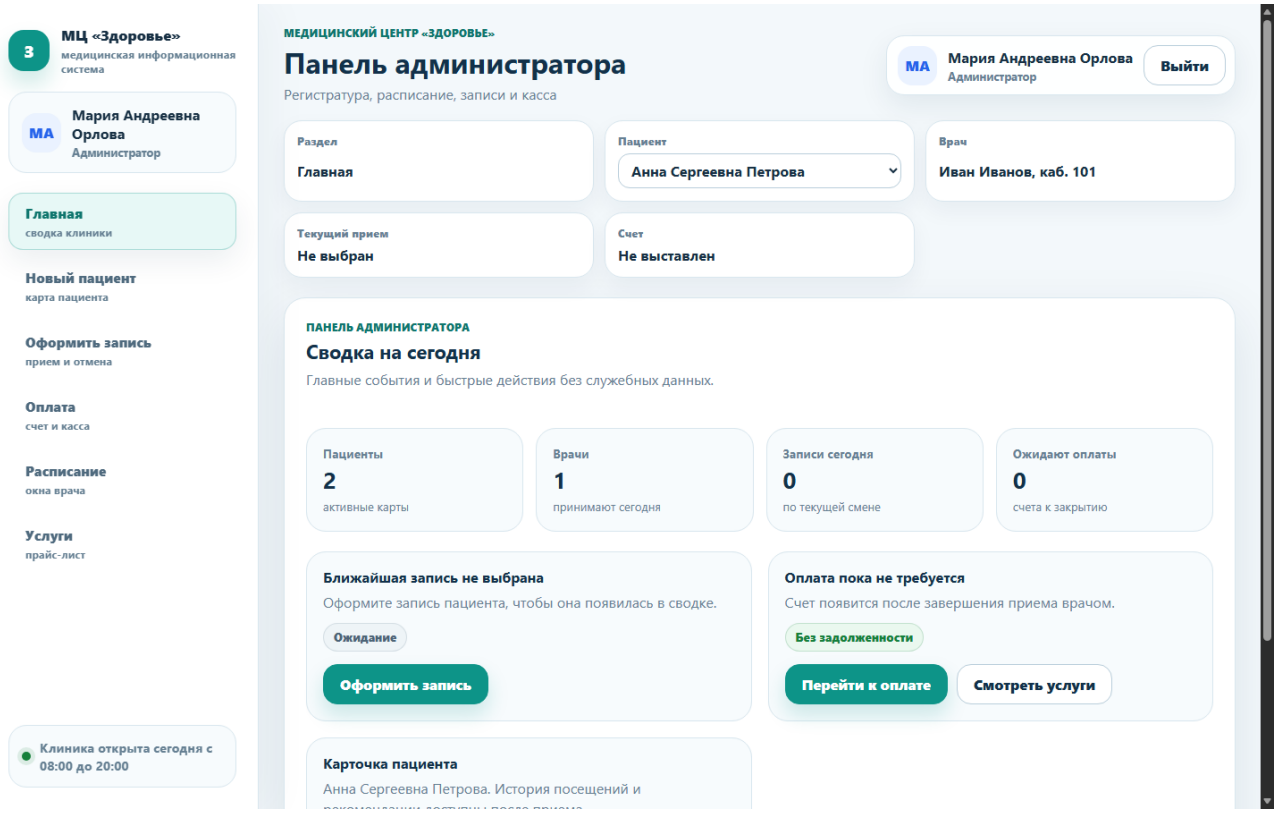


Рисунок 3.4 – Рабочее место пользователя после входа

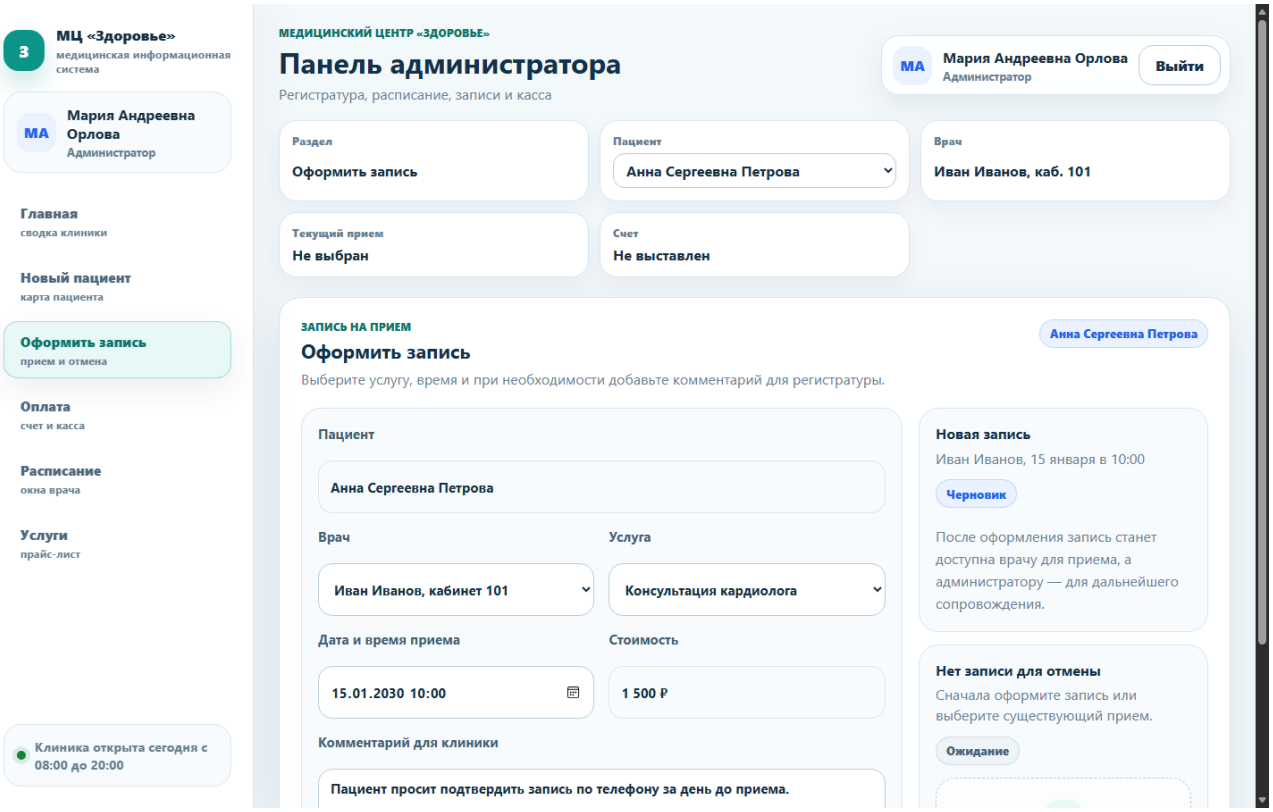


Рисунок 3.5 – Экран оформления записи на прием

Для входной валидации в Go используется пакет go-playground/validator. Входные UUID проверяются на ненулевое значение, телефон проверяется регулярным выражением, e-mail проверяется стандартным валидатором, а дата рождения передается типом, сериализуемым в формат YYYY-MM-DD. Фрагмент репозитория, вызывающего функции PostgreSQL через database/sql, приведен в листинге А.7 приложения А.

3.7 Порядок тестирования и классы эквивалентности

Тестирование выполнялось на трех уровнях: проверка входной валидации backend-приложения, проверка SQL-сценариев базы данных и проверка интеграционных сценариев через REST API. Для базы данных подготовлены сценарии функциональной проверки, конкурентного доступа и производительности.

Классы эквивалентности для функций базы данных приведены в таблице 3.7.

Таблица 3.7 – Классы эквивалентности для функций базы данных

Функция	Корректные классы	Некорректные классы
set_ updated_at	Обновление строки таблицы, содержащей поле updated_at.	Отсутствие поля updated_at в таблице, к которой ошибочно подключен триггер.
has_role	Пользователь имеет требуемую роль; пользователь не имеет требуемую роль.	Несуществующий пользователь или несуществующий код роли.
get_service_ price	Для услуги найдена цена, действующая на заданный момент.	Услуга не имеет активной цены на заданный момент.
book_ appointment	Активный пациент, активный врач, активная услуга, время в будущем, интервал входит в расписание и не пересекается с другой записью.	Прошедшее время, неактивный пациент, неактивный врач, услуга не закреплена за врачом, услуга неактивна, время вне расписания, отсутствие врача, пересечение с другой записью.
cancel_ appointment	Отмена записи со статусом booked; отмена администратором; отмена пациентом более чем за 24 часа.	Несуществующая запись, статус не booked, отмена пациентом менее чем за 24 часа до приема.
complete_ visit	Завершение записи врачом, связанным с приемом; завершение с базовой услугой; завершение с дополнительными оказанными услугами.	Несуществующая запись, другой врач, статус не booked, дополнительная услуга не закреплена за врачом.
register_ payment	Регистрация оплаты с положительной суммой и допустимым способом оплаты.	Несуществующий счет, неположительная сумма, недопустимый способ оплаты.
refresh_ invoice_state	Нет оплат, частичная оплата, полная или избыточная оплата.	Несуществующий счет; некорректные строки счета невозможны при соблюдении ограничений CHECK.
Триггеры пересчета счета	Добавление, изменение или удаление строки счета; добавление, изменение или удаление оплаты.	Ошибочная привязка триггера к таблице без поля invoice_id.

Порядок проверки базы данных включает следующие шаги:

1) создание структуры базы данных из сценария db/schema.sql;
2) загрузку начальных данных из сценария db/seed.sql;
3) выполнение сценария db/test_scenarios.sql, проверяющего регистрацию пациента, валидацию e-mail и телефона, запись, запрет пересечения, отмену, проведение приема, формирование счета и оплату;
4) выполнение двух сценариев конкурентного доступа:
   db/concurrency_session_1.sql;
   db/concurrency_session_2.sql;
5) загрузку массового набора данных из db/bulk_seed.sql и выполнение запросов EXPLAIN ANALYZE из db/performance.sql.

Фрагмент сценария проверки некорректного класса эквивалентности для записи на прием приведен в листинге А.8 приложения А.

На уровне backend выполняются unit-тесты валидаторов, обработки ошибок PostgreSQL, поведения кэша и HTTP-маршрутов. Для frontend выполняется сборка npm run build, подтверждающая корректность TypeScript-кода и клиентского интерфейса.

3.8 Вывод

В технологическом разделе обоснован выбор PostgreSQL, Redis, Go, Gin, React и TypeScript, описаны реализованные сущности, ограничения целостности, функции, триггеры, ролевая модель базы данных, REST-интерфейс доступа к данным и порядок тестирования. Реализация соответствует требованиям задания и использует возможности СУБД для контроля критических правил медицинской клиники.

4 Исследовательский раздел

4.1 Описание исследования

В исследовательском разделе проведено исследование для оценки времени выполнения ключевых операций разработанной базы данных медицинской клиники при увеличении объема данных. Исследование выполнялось на контейнерном стенде, включающем PostgreSQL, Redis, backend-приложение на Go и frontend-приложение на React.

Для проведения исследования использовался сценарий research_benchmark.sql из каталога db. В отличие от вспомогательных проверок, данный сценарий не откатывает изменения, а наполняет рабочую базу данных исследовательским набором записей. Это позволяет проводить замеры на реально заполненных таблицах, которые затем доступны backend-приложению и frontend-интерфейсу.

В ходе исследования последовательно формировались три серии данных: 5000, 20000 и 50000 записей в основных таблицах предметной области. После каждой серии выполнялась команда ANALYZE. Для исследования влияния индексов и оптимизации SQL-запросов выполнялись замеры внутри PostgreSQL: для каждой операции проводилось семь запусков EXPLAIN ANALYZE. В таблицах и графиках приведено среднее время выполнения в миллисекундах. Redis-кэширование исследовалось отдельно как полное время HTTP-запроса к backend-приложению.

Измерялись следующие операции:

— получение расписания врача за период с использованием индексов;
— получение расписания врача при отключенных index scan и bitmap scan;
— получение расписания врача через backend при последовательном добавлении индексов и Redis-кэширования;
— получение истории приемов пациента;
— получение счетов пациента;
— построение отчета по выручке врача;
— вставка новой записи на прием через функцию book_appointment схе-

мы scheduling;

— поиск строк по идентификатору пациента с индексом и без индекса на отдельной исследовательской таблице.

После завершения исследования база данных содержала строки, приведенные в таблице 4.1.

Таблица 4.1 – Количество строк после выполнения исследовательского сценария

Таблица	Количество строк
medical.patients	50002
scheduling.appointments	50008
medical.visits	50000
billing.invoices	50000
billing.invoice_items	50000
billing.payments	50000

4.2 Результаты исследования

Результаты PostgreSQL-замеров приведены в таблице 4.2. Время измерялось в миллисекундах.

Таблица 4.2 – Среднее время выполнения операций

Объем	Расписание	Расписание без index scan	История пациента	Счета пациента	Отчет	Запись
5000	1,717	1,745	0,176	0,129	4,828	0,559
20000	3,387	6,292	0,194	0,403	19,865	0,253
50000	7,614	23,188	0,143	1,325	57,672	0,276

График времени выполнения операций чтения приведен на рисунке 4.1.

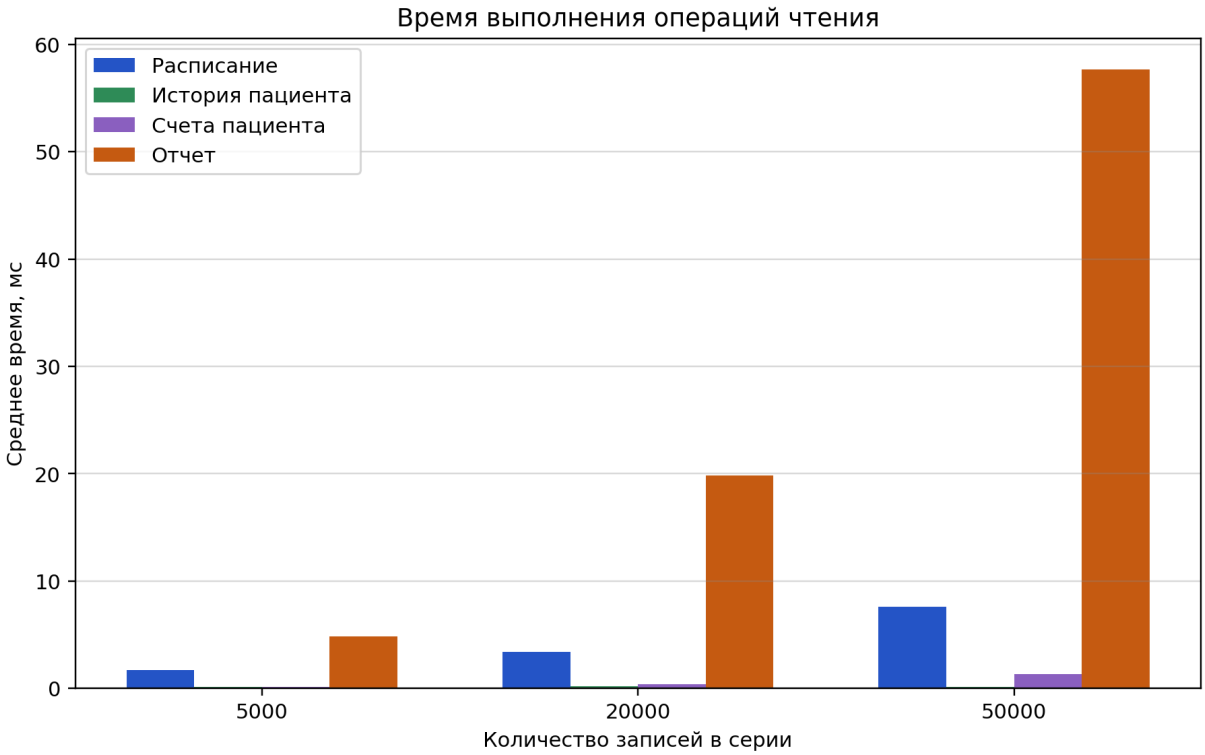


Рисунок 4.1 – Время выполнения операций чтения при разном объеме данных

На рисунке 4.2 показано сравнение получения расписания врача при обычном плане выполнения и при отключенных index scan и bitmap scan.

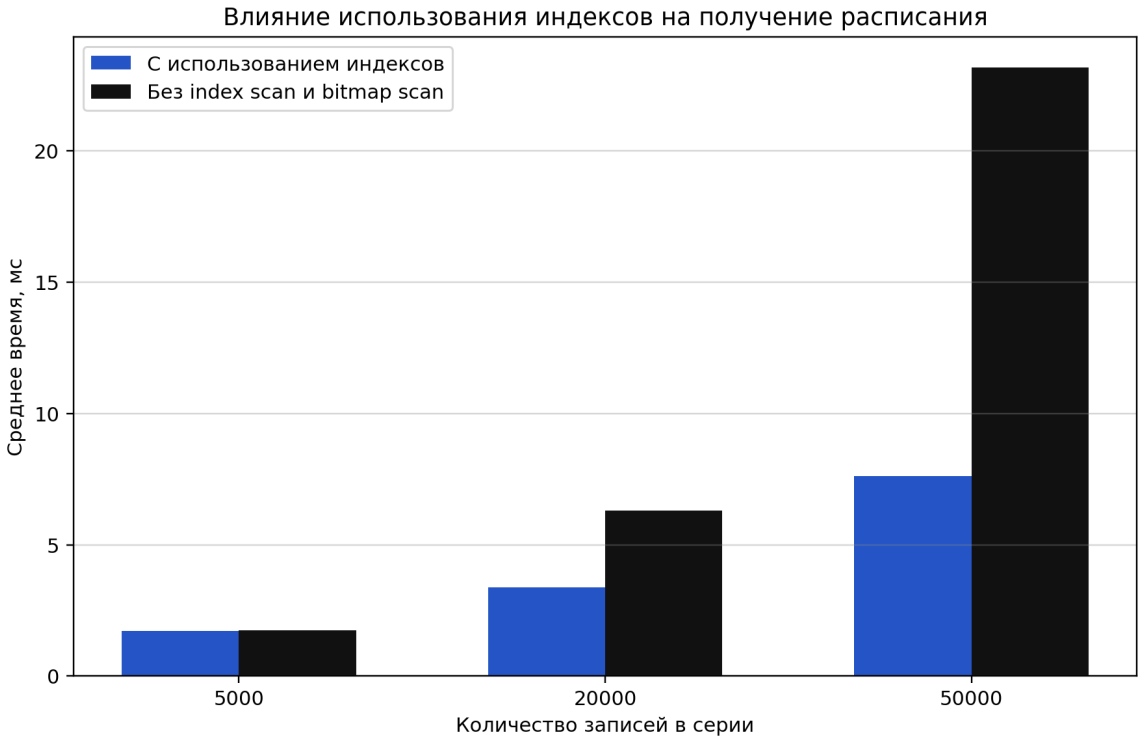


Рисунок 4.2 – Влияние использования индексов на получение расписания врача

Дополнительно были выполнены замеры последовательного ускорения получения расписания врача на уровне полного HTTP-запроса к backend-приложению. Такой замер позволяет сравнить не только время выполнения SQL-запроса внутри PostgreSQL, но и полный путь пользовательского запроса: HTTP-обработку, обращение к базе данных или Redis, сериализацию JSON и передачу ответа.

Для каждого объема данных сравнивались три режима: запрос без Redis-кэша и без обычных B-tree индексов расписания, запрос без Redis-кэша при наличии индексов и повторный запрос из Redis-кэша. Перед запросами без Redis выполнялась очистка ключей Redis, чтобы backend был вынужден читать данные из PostgreSQL. Для режима без индексов временно удалялись индексы `idx_schedule_availability_lookup`, `idx_schedule_absence_lookup` и `idx_appointments_doctor_time`; после замеров индексы создавались обратно. Для каждого режима усреднялись семь запусков. Результаты приведены в таблице 4.3.

Таблица 4.3 – Ускорение полного HTTP-запроса расписания врача

Объем	Без инд., мс	С инд., мс	Redis hit, мс	Индексы, %	Redis, %	Итог, %
5000	36,893	29,719	26,801	19,44	9,82	27,35
20000	149,756	105,388	97,442	29,63	7,54	34,93
50000	292,108	279,806	260,563	4,21	6,88	10,80

На рисунке 4.3 показана цепочка ускорения полного HTTP-запроса: сначала за счет индексов PostgreSQL, затем за счет Redis-кэширования повторного запроса.

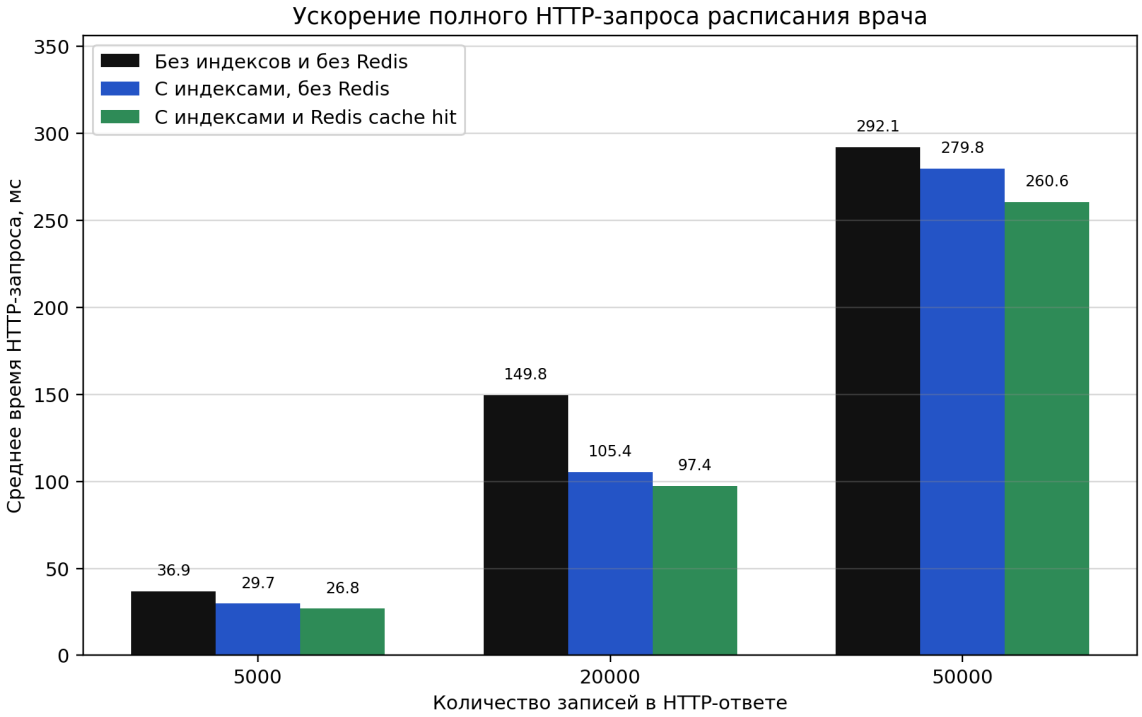


Рисунок 4.3 – Ускорение полного HTTP-запроса расписания врача

График времени вставки новой записи на прием приведен на рисунке 4.4.

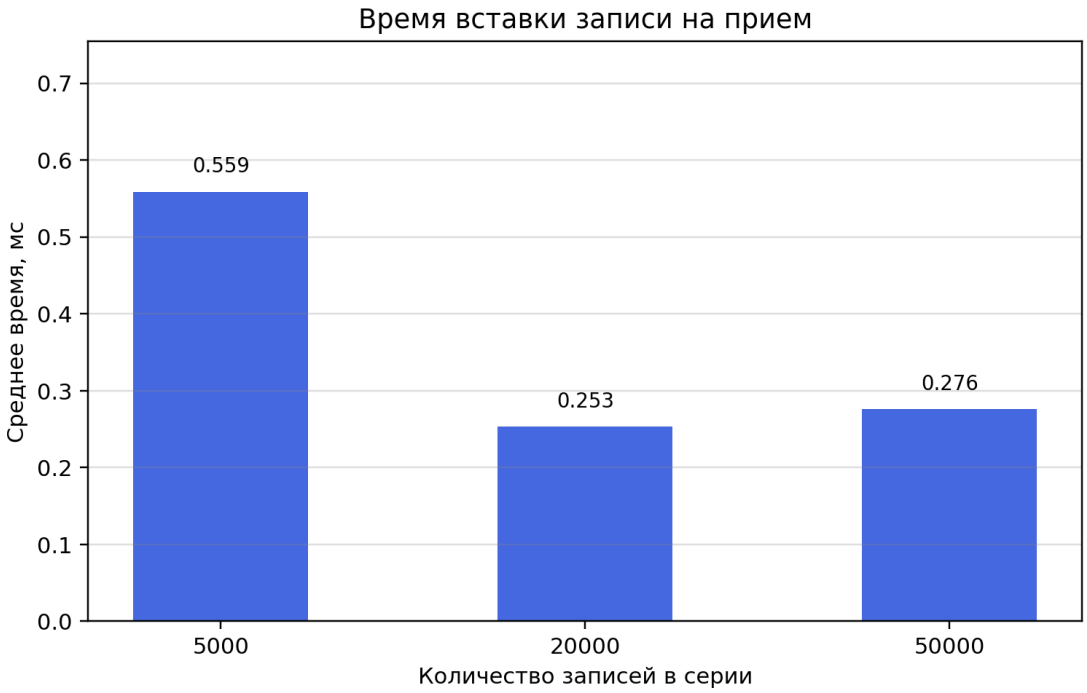


Рисунок 4.4 – Время вставки записи на прием при разном объеме данных

Дополнительное исследование влияния индекса на поиск строк по идентификатору пациента приведено на рисунке 4.5. Для него использовалась отдельная таблица `research_index_search`, которая последовательно на-

полнялась наборами от 1000 до 50000 строк. После каждого наполнения сравнивалось выполнение запроса без индекса и с B-tree индексом по полю `patient_id`; для каждой точки усреднялись пять запусков EXPLAIN ANALYZE.

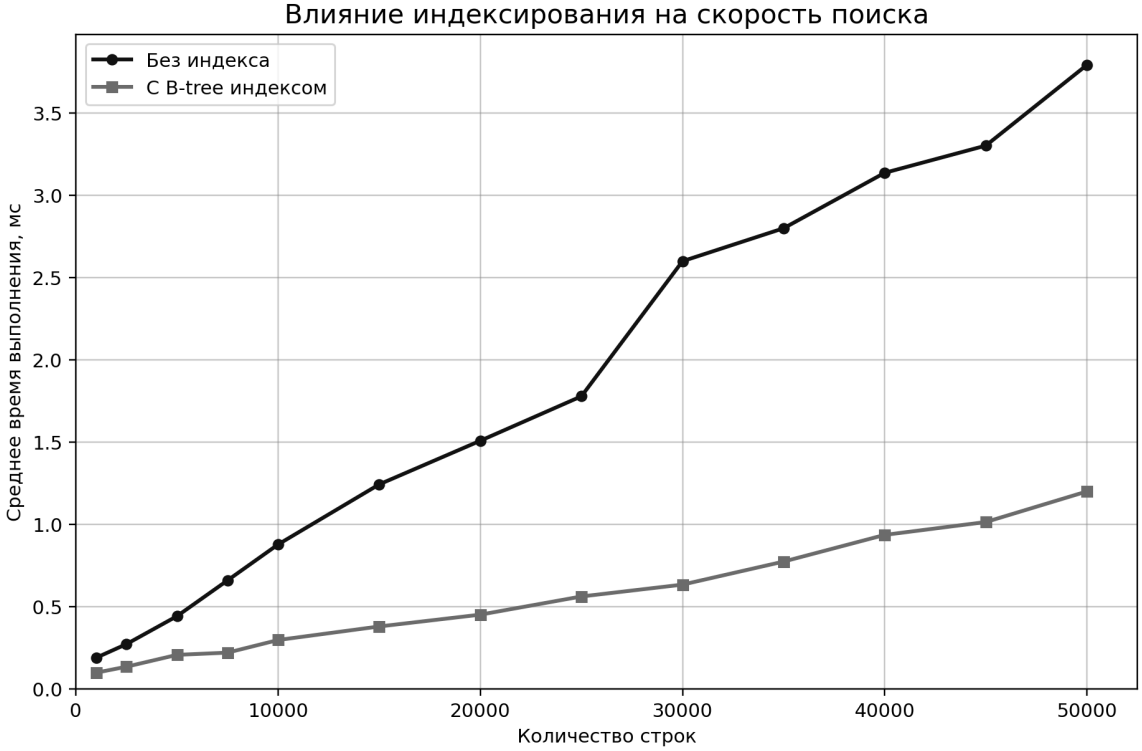


Рисунок 4.5 – Влияние индексирования на поиск по идентификатору пациента

4.3 Анализ результатов

Наибольшее время выполнения показал отчет по выручке врача. При увеличении объема данных с 5000 до 50000 записей среднее время построения отчета выросло с 4,828 мс до 57,672 мс. Это объясняется тем, что запрос соединяет таблицы записей, приемов и счетов, после чего выполняет группировку результата.

Получение расписания врача при использовании индексов заняло 7,614 мс на объеме 50000 записей. При отключенных index scan и bitmap scan время той же операции выросло до 23,188 мс. Таким образом, на максимальном исследованном объеме использование индексов ускорило получение расписания примерно в 3 раза.

Дополнительный HTTP-замер показывает последовательный эффект оптимизаций на уровне всего backend-запроса. На объеме 20000 записей добав-

ление индексов уменьшило среднее время с 149,756 мс до 105,388 мс, то есть на 29,63%. Повторное чтение из Redis-кэша уменьшило время до 97,442 мс. На объеме 50000 записей итоговый выигрыш составил 10,80%: время снизилось с 292,108 мс до 260,563 мс. Меньший выигрыш на максимальном объеме связан с тем, что существенную часть полного HTTP-запроса занимает не только доступ к PostgreSQL, но и сериализация, десериализация и передача большого JSON-ответа.

Запрос истории пациента и запрос счетов пациента выполнялись быстрее агрегирующего отчета. Для истории пациента среднее время на максимальном объеме составило 0,143 мс, для счетов пациента — 1,325 мс. Это связано с тем, что данные запросы используют условия по patient_id и ограничение количества возвращаемых строк.

Вставка записи через функцию scheduling.book_appointment оставалась устойчивой на всех исследованных объемах. На 50000 записей среднее время составило 0,276 мс. Небольшая немонотонность значений связана с тем, что измерения выполняются на долях миллисекунды и зависят от состояния кэша PostgreSQL и планировщика контейнера. Кроме того, функция не просматривает всю таблицу записей, а выполняет проверки активности пациента, врача, услуги, доступности врача и отсутствия пересечения по времени с опорой на индексы и ограничение EXCLUDE.

Дополнительный график поиска по `patient_id` показывает, что при росте числа строк запрос без индекса становится заметно медленнее, тогда как запрос с B-tree индексом растет значительно слабее. На объеме 50000 строк среднее время поиска без индекса составило 3,792 мс, а с индексом — 1,201 мс.

4.4 Вывод

В результате исследования установлено, что разработанная структура базы данных сохраняет приемлемое время выполнения ключевых операций на объеме 50000 записей в основных таблицах. Наиболее чувствительной к росту объема данных является операция построения отчета по выручке, так как она требует соединения и агрегации большого числа строк.

Использование индексов существенно влияет на операции расписания: при 50000 записей отключение index scan увеличило время получения распи-

сания с 7,614 мс до 23,188 мс. Дополнительный замер поиска по `patient_id` также показал преимущество индексированного доступа. Операция записи пациента на прием через функцию базы данных осталась в пределах 0,276 мс на максимальном объеме, что подтверждает корректность выбранной стратегии согласованности: транзакционная функция PostgreSQL, индексы и EXCLUDE-ограничение позволяют предотвращать конфликтующие записи без существенного роста времени вставки.

Дополнительное исследование полного HTTP-запроса показало, что последовательное добавление индексов и Redis-кэширования уменьшает время получения расписания врача. При этом величина выигрыша зависит от размера ответа: на больших объемах данных ограничивающим фактором становится не только доступ к PostgreSQL, но и сериализация результата. Поэтому Redis целесообразно использовать для часто повторяющихся запросов расписания и прайс-листа, а также совместно с ограничением размера возвращаемого периода.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы достигнута поставленная цель: спроектированы и реализованы база данных медицинской клиники и приложение доступа к ней, обеспечивающие корректную запись на прием, проведение приема, формирование счетов, учет оплат и кэширование часто используемых данных.

При выполнении работы получены следующие основные результаты:

1) проведен анализ предметной области медицинской клиники и формализована задача разработки базы данных и приложения;
2) спроектированы сущности базы данных, связи между ними, ограничения целостности, ролевая модель, процедуры и триггеры;
3) выбраны средства реализации, соответствующие требованиям задания: PostgreSQL, Redis, Go, Gin, React и TypeScript;
4) реализованы таблицы, индексы, домены, перечислимые типы, хранимые процедуры, триггеры и прикладной REST API;
5) разработаны тестовые сценарии для проверки записи, отмены, проведения приема, формирования счетов и регистрации оплаты;
6) исследованы конкурентный доступ и влияние индексов на производительность ключевых операций.

Поставленные задачи решены в полном объеме: разработаны структура базы данных, ролевая модель, функции, триггеры, REST API, frontend-интерфейс и внешнее кэширование с использованием Redis.

Практическая проверка показала, что система запрещает конфликтующие записи по времени, корректно выполняет основные бизнес-операции и ускоряет чтение расписания и прайс-листа за счет кэширования. Результаты исследования подтвердили приемлемое время выполнения ключевых операций на выбранных объемах данных.

Разработанная система может использоваться как основа информационной системы небольшого медицинского центра. Для дальнейшего развития рекомендуется добавить журнал аудита действий пользователей, расширить управление расписанием нескольких врачей и реализовать резервное копирование базы данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Дейт К. Дж. Введение в системы баз данных / К. Дж. Дейт. — 8-е изд. — М. : Вильямс, 2005. — 1328 с.

2. Коннолли Т. Базы данных: проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. — 3-е изд. — М. : Вильямс, 2003. — 1436 с.

3. PostgreSQL: Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/ (дата обращения: 01.04.2026).

4. PostgreSQL: Range Types [Электронный ресурс]. — URL: https://www.postgresql.org/docs/current/rangetypes.html (дата обращения: 01.04.2026).

5. PostgreSQL: Exclusion Constraints [Электронный ресурс]. — URL: https://www.postgresql.org/docs/current/ddl-constraints.html (дата обращения: 01.04.2026).

6. Redis: Documentation [Электронный ресурс]. — URL: https://redis.io/docs/latest/ (дата обращения: 01.04.2026).

7. Go: Documentation [Электронный ресурс]. — URL: https://go.dev/doc/ (дата обращения: 01.04.2026).

8. Gin Web Framework: Documentation [Электронный ресурс]. — URL: https://gin-gonic.com/ (дата обращения: 01.04.2026).

9. React: Documentation [Электронный ресурс]. — URL: https://react.dev/ (дата обращения: 01.04.2026).

10. TypeScript: Documentation [Электронный ресурс]. — URL: https://www.typescriptlang.org/docs/ (дата обращения: 01.04.2026).

ПРИЛОЖЕНИЕ А

```sql
CREATE TABLE IF NOT EXISTS scheduling.appointments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES medical.patients(id) ON DELETE RESTRICT,
    doctor_id UUID NOT NULL REFERENCES medical.doctors(id) ON DELETE RESTRICT,
    booked_service_id UUID NOT NULL REFERENCES catalog.services(id) ON DELETE
        RESTRICT,
    starts_at TIMESTAMPTZ NOT NULL,
    ends_at TIMESTAMPTZ NOT NULL,
    status scheduling.appointment_status NOT NULL DEFAULT 'booked',
    note TEXT,
    cancellation_reason TEXT,
    created_by_user_id UUID NOT NULL REFERENCES security.users(id) ON DELETE
        RESTRICT,
    cancelled_by_user_id UUID REFERENCES security.users(id) ON DELETE RESTRICT,
    completed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    period TSTZRANGE GENERATED ALWAYS AS (tstzrange(starts_at, ends_at, '[)'))
        STORED,
    CHECK (ends_at > starts_at)
);
```

Листинг А.1 – Фрагмент реализации сущности записи на прием

```sql
CREATE DOMAIN security.email_address AS VARCHAR(150)
    CHECK (VALUE IS NULL OR VALUE ~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$');

CREATE DOMAIN security.phone_number AS VARCHAR(20)
    CHECK (VALUE IS NULL OR VALUE ~ '^\+?[1-9][0-9]{10,14}$');

ALTER TABLE catalog.service_prices
    ADD CONSTRAINT service_prices_no_overlap
    EXCLUDE USING gist (service_id WITH =, period WITH &&);

ALTER TABLE scheduling.doctor_availability
    ADD CONSTRAINT doctor_availability_no_overlap
    EXCLUDE USING gist (doctor_id WITH =, period WITH &&)
    WHERE (is_active);

ALTER TABLE scheduling.appointments
    ADD CONSTRAINT appointments_doctor_no_overlap
    EXCLUDE USING gist (doctor_id WITH =, period WITH &&)
    WHERE (status IN ('booked', 'completed'));
```

Листинг А.2 – Фрагменты реализации ограничений целостности

```
CREATE OR REPLACE FUNCTION catalog.get_service_price(
    p_service_id UUID,
    p_at TIMESTAMPTZ
)
RETURNS NUMERIC(12,2)
LANGUAGE plpgsql
AS $$
DECLARE
    v_price NUMERIC(12,2);
BEGIN
    SELECT sp.price
      INTO v_price
      FROM catalog.service_prices sp
     WHERE sp.service_id = p_service_id
       AND sp.effective_from <= p_at
       AND (sp.effective_to IS NULL OR sp.effective_to > p_at)
     ORDER BY sp.effective_from DESC
     LIMIT 1;

    IF v_price IS NULL THEN
        RAISE EXCEPTION USING ERRCODE = '22023',
            MESSAGE = format('Price is not found for service %s at %s',
                             p_service_id,
                             p_at);
    END IF;

    RETURN v_price;
END;
$$;
```

Листинг А.3 – Функция получения актуальной цены услуги

```
CREATE OR REPLACE FUNCTION scheduling.book_appointment(
    p_patient_id UUID,
    p_doctor_id UUID,
    p_service_id UUID,
    p_starts_at TIMESTAMPTZ,
    p_created_by_user_id UUID,
    p_note TEXT DEFAULT NULL
)
RETURNS UUID
LANGUAGE plpgsql
AS $$
DECLARE
    v_duration INTEGER;
```

```
    v_ends_at TIMESTAMPTZ;
    v_appointment_id UUID;
BEGIN
    IF p_starts_at <= NOW() THEN
        RAISE EXCEPTION USING ERRCODE = '22023',
            MESSAGE = 'Appointment start time must be in the future';
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM medical.patients
         WHERE id = p_patient_id
           AND status = 'active'
    ) THEN
        RAISE EXCEPTION USING ERRCODE = '22023',
            MESSAGE = format('Patient %s is inactive', p_patient_id);
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM medical.doctors
         WHERE id = p_doctor_id
           AND status = 'active'
    ) THEN
        RAISE EXCEPTION USING ERRCODE = '22023',
            MESSAGE = format('Doctor %s is inactive', p_doctor_id);
    END IF;

    IF NOT EXISTS (
        SELECT 1
          FROM medical.doctor_services
         WHERE doctor_id = p_doctor_id
           AND service_id = p_service_id
    ) THEN
        RAISE EXCEPTION USING ERRCODE = '22023',
            MESSAGE = format('Service %s is not assigned to doctor %s',
                             p_service_id,
                             p_doctor_id);
    END IF;

    SELECT duration_minutes
      INTO v_duration
      FROM catalog.services
     WHERE id = p_service_id
       AND status = 'active';

    IF v_duration IS NULL THEN
        RAISE EXCEPTION USING ERRCODE = '22023',
            MESSAGE = format('Service %s is inactive', p_service_id);
    END IF;
```

```
62
63      v_ends_at := p_starts_at + make_interval(mins => v_duration);
64
65      IF NOT EXISTS (
66          SELECT 1
67            FROM scheduling.doctor_availability da
68           WHERE da.doctor_id = p_doctor_id
69             AND da.is_active
70             AND da.starts_at <= p_starts_at
71             AND da.ends_at >= v_ends_at
72      ) THEN
73          RAISE EXCEPTION USING ERRCODE = '22023',
74              MESSAGE = 'Time is outside doctor availability';
75      END IF;
76
77      INSERT INTO scheduling.appointments (
78          patient_id,
79          doctor_id,
80          booked_service_id,
81          starts_at,
82          ends_at,
83          status,
84          note,
85          created_by_user_id
86      )
87      VALUES (
88          p_patient_id,
89          p_doctor_id,
90          p_service_id,
91          p_starts_at,
92          v_ends_at,
93          'booked',
94          p_note,
95          p_created_by_user_id
96      )
97      RETURNING id INTO v_appointment_id;
98
99      RETURN v_appointment_id;
100 END;
101 $$;
```

Листинг А.4 – Фрагмент функции записи на прием

```
1  CREATE OR REPLACE FUNCTION billing.trg_refresh_invoice_from_items()
2  RETURNS trigger
3  LANGUAGE plpgsql
4  AS $$
5  BEGIN
```

```
    PERFORM billing.refresh_invoice_state(COALESCE(NEW.invoice_id,
        OLD.invoice_id));
    RETURN COALESCE(NEW, OLD);
END;
$$;

CREATE TRIGGER trg_invoice_items_refresh_invoice
AFTER INSERT OR UPDATE OR DELETE ON billing.invoice_items
FOR EACH ROW
EXECUTE FUNCTION billing.trg_refresh_invoice_from_items();

CREATE TRIGGER trg_payments_refresh_invoice
AFTER INSERT OR UPDATE OR DELETE ON billing.payments
FOR EACH ROW
EXECUTE FUNCTION billing.trg_refresh_invoice_from_payments();
```

Листинг А.5 – Триггеры пересчета состояния счета

```
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'clinic_owner') THEN
        CREATE ROLE clinic_owner NOLOGIN;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'clinic_api') THEN
        CREATE ROLE clinic_api NOLOGIN;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'clinic_reporting')
       THEN
        CREATE ROLE clinic_reporting NOLOGIN;
    END IF;
END
$$;

GRANT USAGE ON SCHEMA security, catalog, medical, scheduling, billing
TO clinic_api, clinic_reporting;

GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES
IN SCHEMA security, catalog, medical, scheduling, billing
TO clinic_api;

GRANT EXECUTE ON ALL FUNCTIONS
IN SCHEMA security, catalog, medical, scheduling, billing
TO clinic_api;

GRANT SELECT ON ALL TABLES
IN SCHEMA security, catalog, medical, scheduling, billing
```

```
TO clinic_reporting;
```

Листинг А.6 – Реализация ролей базы данных

```
func (r *AppointmentRepository) Book(
    ctx context.Context,
    command model.BookAppointmentCommand,
) (uuid.UUID, error) {
    var appointmentID uuid.UUID
    const query = "SELECT scheduling.book_appointment($1, $2, $3, " +
        "$4, $5, $6)"
    err := r.db.QueryRowContext(ctx, query,
        command.PatientID,
        command.DoctorID,
        command.ServiceID,
        command.StartsAt,
        command.CreatedByUserID,
        nullableString(command.Note),
    ).Scan(&appointmentID)

    return appointmentID, err
}

func (r *AppointmentRepository) RegisterPayment(
    ctx context.Context,
    command model.RegisterPaymentCommand,
) (uuid.UUID, error) {
    var paymentID uuid.UUID
    const query = "SELECT billing.register_payment($1, $2, $3, $4, $5)"
    err := r.db.QueryRowContext(ctx, query,
        command.InvoiceID,
        command.Amount,
        string(command.PaymentMethod),
        command.ReceivedByUserID,
        nullableString(command.ExternalReference),
    ).Scan(&paymentID)

    return paymentID, err
}
```

Листинг А.7 – Вызов функций PostgreSQL из репозитория

```
BEGIN
    PERFORM scheduling.book_appointment(
        v_patient_2,
        v_doctor,
        v_service_consult,
        TIMESTAMPTZ '2030-01-15 10:15:00+03',
```

```
        v_patient_user,
        'SCENARIO_OVERLAP_REJECT'
    );
    v_rejected := FALSE;
EXCEPTION WHEN OTHERS THEN
    v_rejected := TRUE;
END;

IF NOT v_rejected THEN
    RAISE EXCEPTION 'Overlapping booking was not rejected';
END IF;
```

Листинг А.8 – Проверка запрета пересечения записей

К расчётно-пояснительной записке курсовой работы прилагаются следующие материалы:

— презентация к курсовой работе, состоящая из 21 слайда;

— USB-флеш-накопитель с исходными кодами, электронными версиями РПЗ и презентации.